

Lab3-driver

章尹睿

Outline

- Lab3 主旨
- Zedboard 開機流程
- SD 卡內容介紹
- Main Flow
 - 環境建置
 - 創建專案
 - 產生開機檔
 - 如何修改Driver(補充)
 - 應用程式說明(補充)
 - SD卡記憶體切割
 - 產生系統碟到SD卡
 - 複製開機檔到SD卡
 - SD卡帳號設定
 - SD卡開機 – 上板

Lab3 主旨

在 Zedboard 上建立一個可執行自訂硬體 (PL) 與軟體 (PS) 整合系統的完整流程

◆ 透過 PetaLinux 整合出一個可開機、可控制硬體的嵌入式系統

PetaLinux –

1. 產生 kernel + device tree
2. 整合 FPGA bitstream (from Vivado)
3. 建立 root filesystem
4. 建立 BOOT.BIN / image.ub (初始化與驅動/運行硬體之資訊)

◆ 沒有 Linux : bare-metal C + 手動寫 register address + interrupt 要自己設定(難擴展、難 debug)

Zedboard 開機流程

➤ Stage 0 : BootROM 初始化

當電力開啟或系統重置 (Reset) 時觸發，ARM 核心會先執行預存在 BootROM 中的初始化代碼。

➤ Stage 1 : FSBL (First Stage Boot Loader)

PS (Processing System) 初始化： 即 ARM SoC 部分。

PL (Programmable Logic) 配置： FSBL 會在啟動裝置 (如 SD 卡) 中的硬體設計檔 (bit file) 寫入 PL，完成 FPGA 的硬體電路配置。

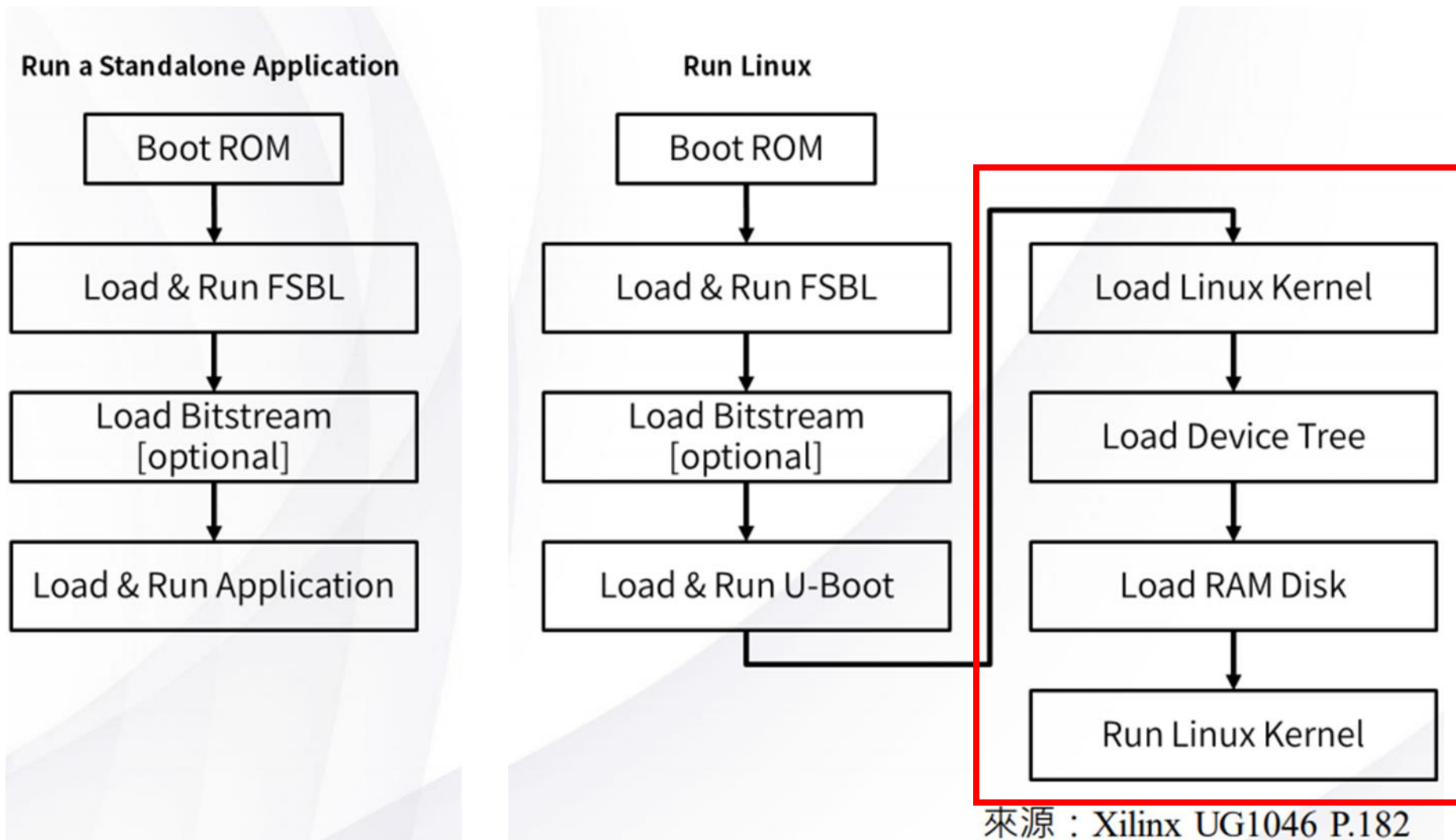
準備記憶體： 初始化外部 RAM，並載入下階段程式 (SSBL 或 stand-alone(bare-metal) application) 。

➤ Stage 2 : SSBL(Second Stage Boot Loader) U-Boot (嵌入式 Linux)

環境設定： U-Boot 會在 CPU0 上執行，建立並設定好 Linux 核心啟動所需的環境。

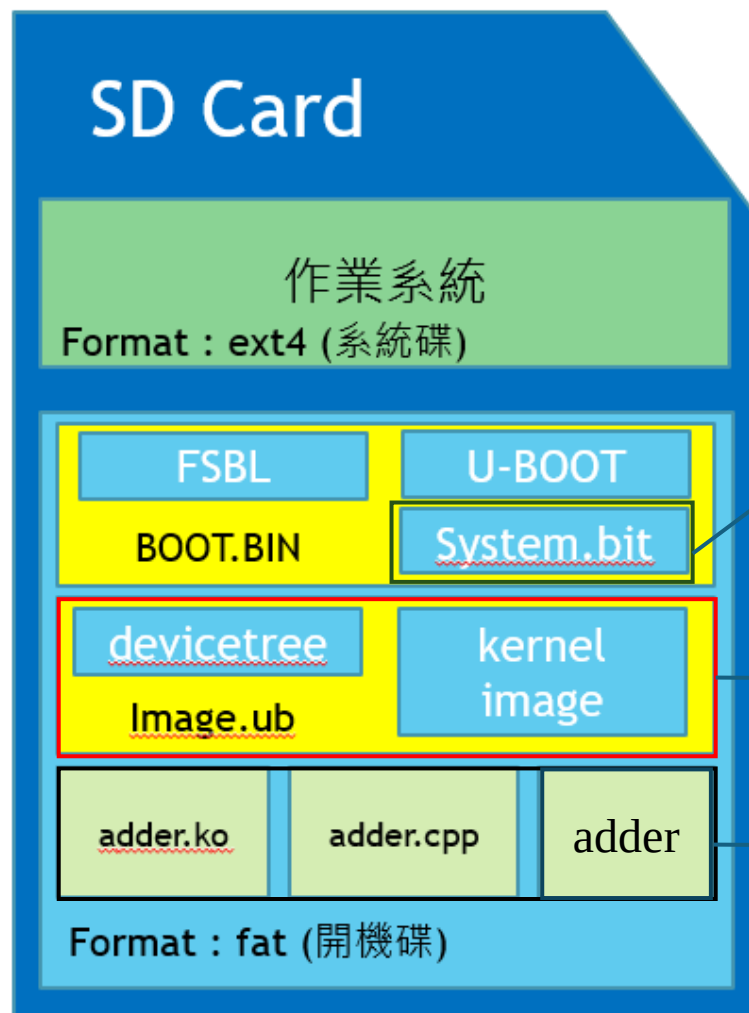
載入系統： 將 Linux 核心影像 (Kernel image)、裝置樹 (Device Tree) 及其他啟動參數載入到 RAM 中，最後將系統的控制權移交給 Linux 核心。

Zedboard 開機流程



來源：Xilinx UG1046 P.182

SD 卡內容介紹



- FSBL.elf : First Stage Bootloader
- U-Boot.elf : Second Stage Bootloader
- System.bit : programming information for an FPGA
- Devicetree.deb : 描述底層硬體相關資訊 (DTB, Devicetree Binary File)

.bit : setting PL bit file, from vivado

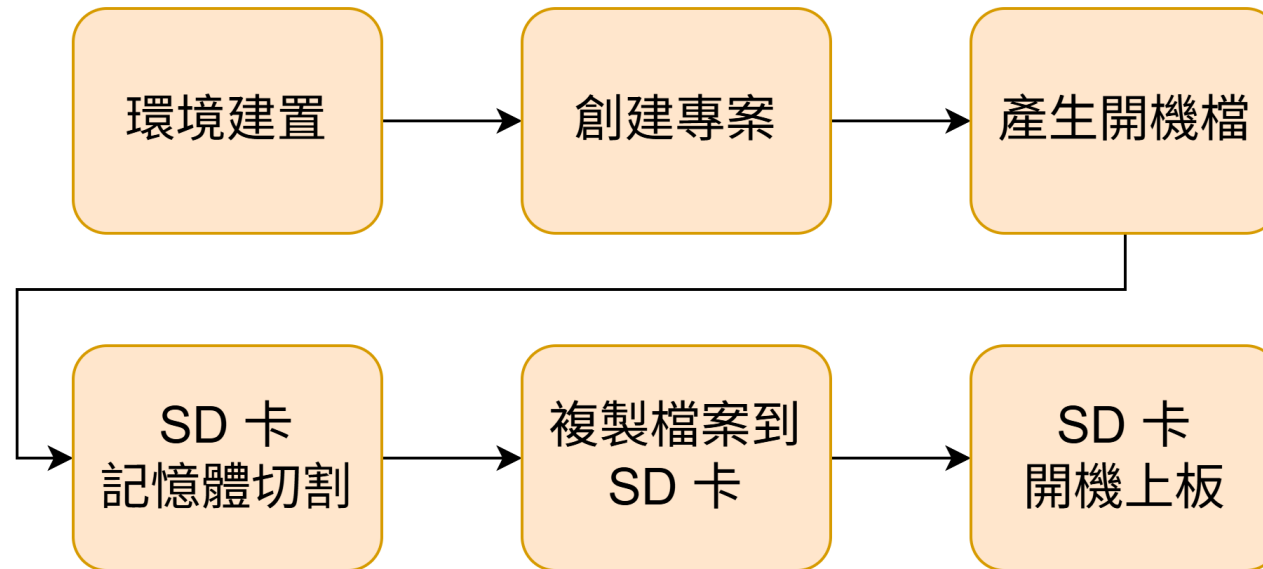
Image.ub : Linux 運行的必要資訊

使用者在 Linux 系統中操作自定義的硬體所需檔案
.ko : compiled driver module , 稍後Lab會以command – insmod 存取他
adder = 編譯檔

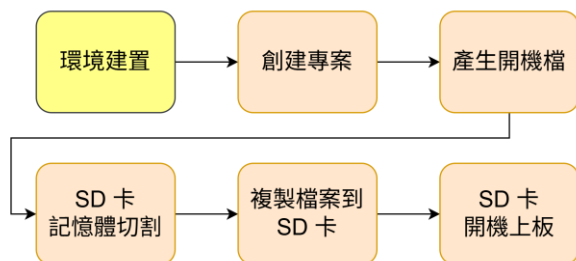
Main Flow

- 同學須先建立好VMware虛擬機環境設定(Ubuntu16.04)與虛擬機的共用資料夾設定且先 copy lab3 所需之資料
- 實驗內容
 - 環境建置
 - 創建專案
 - 產生開機檔
 - 修改Driver & adder 應用程式說明
 - SD卡記憶體切割
 - 產生系統碟到SD卡
 - 複製開機檔到SD卡
 - SD卡帳號設定
 - SD卡開機 – 上板

Main Flow



環境建置



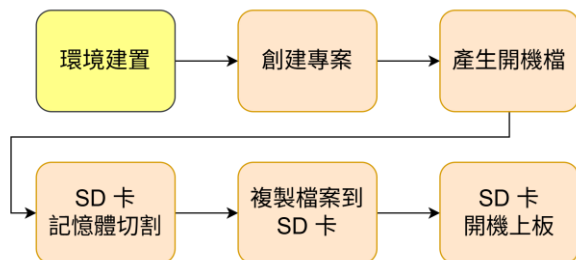
- 其中一個檔案是建環境的command
- 先\$cd Desktop
- \$ cat **env.txt**
- 複製terminal印出來的command執行

```
ta@ubuntu: ~/Desktop
zlib1g-dev is already the newest version (1:1.2.8.dfsg-2ubuntu4.3).
zlib1g:i386 is already the newest version (1:1.2.8.dfsg-2ubuntu4.3).
0 upgraded, 0 newly installed, 0 to remove and 47 not upgraded.
ta@ubuntu:~/Desktop$ cat S0PC\ Driver\ Lab\env.txt
sudo apt-get install -y gcc git make net-tools libncurses5-dev tftpd zlib1g-dev
libssl-dev flex bison libselinux1 gnupg wget diffstat chrpath socat xterm autoc
nf libtool tar unzip texinfo zlib1g-dev gcc-multilib build-essential libstdl1.2-d
ev libglib2.0-dev zlib1g:i386 screen pax gzip gawk
ta@ubuntu:~/Desktop$ sudo apt-get install -y gcc git make net-tools libncurses5-
dev tftpd zlib1g-dev libssl-dev flex bison libselinux1 gnupg wget diffstat chrpa
th socat xterm autoconf libtool tar unzip texinfo zlib1g-dev gcc-multilib build-
essential libstdl1.2-dev libglib2.0-dev zlib1g:i386 screen pax gzip gawk
Reading package lists... Done
Building dependency tree
Reading state information... Done
autoconf is already the newest version (2.69-9).
bison is already the newest version (2:3.0.4.dfsg-1).
build-essential is already the newest version (12.1ubuntu2).
diffstat is already the newest version (1.61-1).
flex is already the newest version (2.6.0-11).
gawk is already the newest version (1:4.1.3+dfsg-0.1).
gcc is already the newest version (4:5.3.1-1ubuntu1).
gcc-multilib is already the newest version (4:5.3.1-1ubuntu1).
gzip is already the newest version (1.6-4ubuntu1).
```

複製結果

貼上結果到
terminal執行

環境建置



- 修改權限及安裝 Petalinux
- `chmod +x petalinux-v2018.3-final-installer.run`
- `./petalinux-v2018.3-final-installer.run ~/build/`

```
ta@ubuntu:~/Desktop$ chmod +x petalinux-v2018.3-final-installer-003.run
ta@ubuntu:~/Desktop$ ./petalinux-v2018.3-final-installer-003.run ~/build/
INFO: Checking installation environment requirements...
```

- 接著先按enter同意, q跟y交錯按
- 安裝的過程要等一下

```
ta@ubuntu: ~/Desktop
INFO: Checking installed development libraries
INFO: Checking network and other services
WARNING: No tftp server found - please refer to "PetaLinux SDK Installation Guide" for its impact and solution
INFO: Checking installer checksum...
INFO: Extracting PetaLinux installer...

LICENSE AGREEMENTS

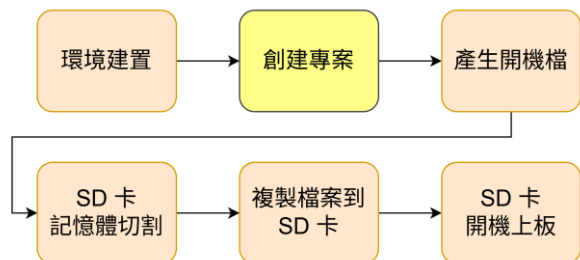
PetaLinux SDK contains software from a number of sources. Please review the following licenses and indicate your acceptance of each to continue.

You do not have to accept the licenses, however if you do not then you may not use PetaLinux SDK.

Use PgUp/PgDn to navigate the license viewer, and press 'q' to close

Press Enter to display the license agreements
Do you accept Xilinx End User License Agreement? [y/N] > y
Do you accept Webtalk Terms and Conditions? [y/N] > y
Do you accept Webtalk Terms and Conditions? [y/N] > y
Do you accept Third Party End User License Agreement? [y/N] > y
INFO: Installing PetaLinux...
```

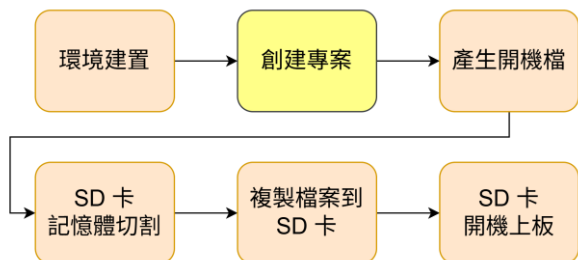
創建專案



- 輸入指令：啟動 PetaLinux 工具環境
- `source ~/build/settings.sh`

```
ta@ubuntu:~/Desktop$ source ~/build/settings.sh
PetaLinux environment set to '/home/ta/build'
WARNING: /bin/sh is not bash!
bash is PetaLinux recommended shell. Please set your default shell to bash.
INFO: Checking free disk space
INFO: Checking installed tools
INFO: Checking installed development libraries
INFO: Checking network and other services
WARNING: No tftp server found - please refer to "PetaLinux SDK Installation Guide" for its impact and solution
ta@ubuntu:~/Desktop$
```

創建專案



- 使用現成的BSP(board support package)建立一個專案，輸入指令：

`petalinux-create -t project -s SOPC_Driver_Lab/avnet-digilent-zedboard-v2018.3-final.bsp`

BSP: 該硬體平台的驅動程式、libraries、Bootloader、設定檔等，以便於在該硬體上運行軟體。

```
ta@ubuntu: ~/Desktop

EXAMPLES:

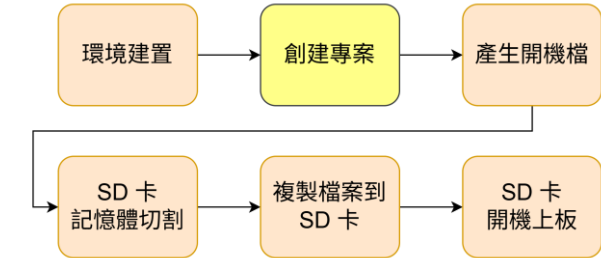
Create project from PetaLinux Project BSP:
$ petalinux-create -t project -s <PATH_TO_PETALINUX_PROJECT_BSP>

Create project from template:
For microblaze project,
$ petalinux-create -t project -n <PROJECT> --template microblaze
For zynq project,
$ petalinux-create -t project -n <PROJECT> --template zynq
For zynqMP project,
$ petalinux-create -t project -n <PROJECT> --template zynqMP

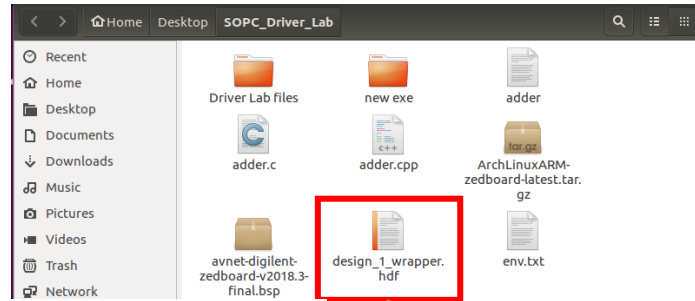
ta@ubuntu:~/Desktop$ petalinux-create -t project -s SOPC_Driver_Lab/avnet-digilent-zedboard-v2018.3-final.bsp
INFO: Create project:
Project /home/ta/Desktop//avnet-digilent-zedboard-2018.3 already exists. Please
input "y" to overwrite: y
INFO: Projects:
INFO: * avnet-digilent-zedboard-2018.3
INFO: has been successfully installed to /home/ta/Desktop/
INFO: New project successfully created in /home/ta/Desktop/
ta@ubuntu:~/Desktop$
```

The terminal screenshot shows the execution of the `petalinux-create` command. It displays examples for creating projects from BSPs or templates, and then shows the successful creation of a project named 'avnet-digilent-zedboard-2018.3' in the directory '/home/ta/Desktop/'.

創建專案



- 取得硬體描述檔(.hdf，此檔案須透過Vivado匯出)，並更新Petalinux專案，輸入以下指令：
- `cd avnet-digilent-zedboard-v2018.3`
- `petalinux-config --get-hw-description=/home/ta/Desktop/SOPC_Driver_Lab`



SD Card

作業系統

Format : ext4 (系統碟)

FSBL

U-BOOT

BOOT.BIN

System.bit

devicetree

kernel image

Image.ub

adder.ko

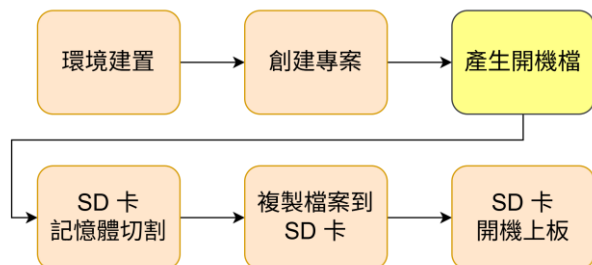
adder.cpp

adder

Format : fat (開機碟)

```
ta@ubuntu: ~/Desktop/avnet-digilent-zedboard-2018.3
ta@ubuntu:~/Desktop$ source ~/build/settings.sh
PetaLinux environment set to '/home/ta/build'
WARNING: /bin/sh is not bash!
bash is PetaLinux recommended shell. Please set your default shell to bash.
INFO: Checking free disk space
INFO: Checking installed tools
INFO: Checking installed development libraries
INFO: Checking network and other services
WARNING: No tftp server found - please refer to "PetaLinux SDK Installation Guide" for its impact and solution
ta@ubuntu:~/Desktop$ petalinux-create -t project -s SOPC_Driver_Lab/avnet-digilent-zedboard-v2018.3-final.bsp
INFO: Create project:
Project /home/ta/Desktop//avnet-digilent-zedboard-2018.3 already exists. Please input "y" to overwrite: y
INFO: Projects:
INFO: * avnet-digilent-zedboard-2018.3
INFO: has been successfully installed to /home/ta/Desktop/
INFO: New project successfully created in /home/ta/Desktop/
ta@ubuntu:~/Desktop$ cd avnet-digilent-zedboard-2018.3
ta@ubuntu:~/Desktop/avnet-digilent-zedboard-2018.3$ petalinux-config --get-hw-description=/home/ta/Desktop/SOPC_Driver_Lab
```

產生開機檔



- 選”Image Packaging Configuration”
- 按enter

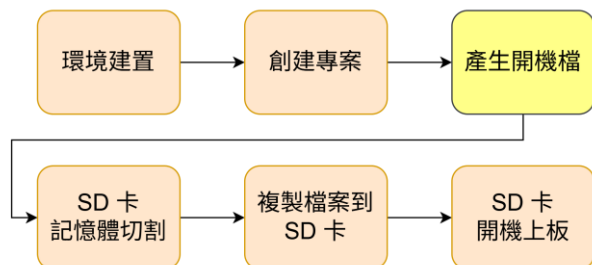
```
dip_lab@dip: ~/Desktop/avnet-digilent-zedboard-2018.3
/home/dip_lab/Desktop/avnet-digilent-zedboard-2018.3/project-spec/configs/config - misc/config System Configuration

misc/config System Configuration
Arrow keys navigate the menu. <Enter> selects submenus ---> (or empty submenus ----).
Highlighted letters are hotkeys. Pressing <Y> includes, <N> excludes, <M> modularizes
features. Press <Esc><Esc> to exit, <?> for Help, </> for Search. Legend: [*] built-in
[ ] excluded <M> module < > module capable

-- ZYNQ Configuration
   Linux Components Selection --->
   Auto Config Settings --->
-- Subsystem AUTO Hardware Settings --->
   DTG Settings --->
   u-boot Configuration --->
   Image Packaging Configuration --->
   Firmware Version Configuration --->
   Yocto Settings --->

<Select> < Exit > < Help > < Save > < Load >
```

產生開機檔



- Root Filesystem 放在SD card內 (image.ub) · 因此需修改”Root filesystem type”
- 按enter

```
dip_lab@dip: ~/Desktop/avnet-digilent-zedboard-2018.3
/home/dip_lab/Desktop/avnet-digilent-zedboard-2018.3/project-spec/configs/config - misc/config Syst
e→ Image Packaging Configuration

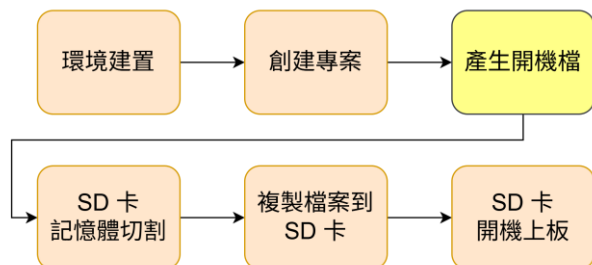
Image Packaging Configuration
Arrow keys navigate the menu. <Enter> selects submenus ---> (or empty submenu ----).
Highlighted letters are hotkeys. Pressing <Y> includes, <N> excludes, <M> modularizes
features. Press <Esc><Esc> to exit, <?> for Help, </> for Search. Legend: [*] built-in
[ ] excluded <M> module < > module capable

  Root filesystem type (INITRAMFS) ---->
(image.ub) name for bootable kernel image
(0x1000) DTB padding size
[*] Copy final images to tftpboot
(/tftpboot) tftpboot directory

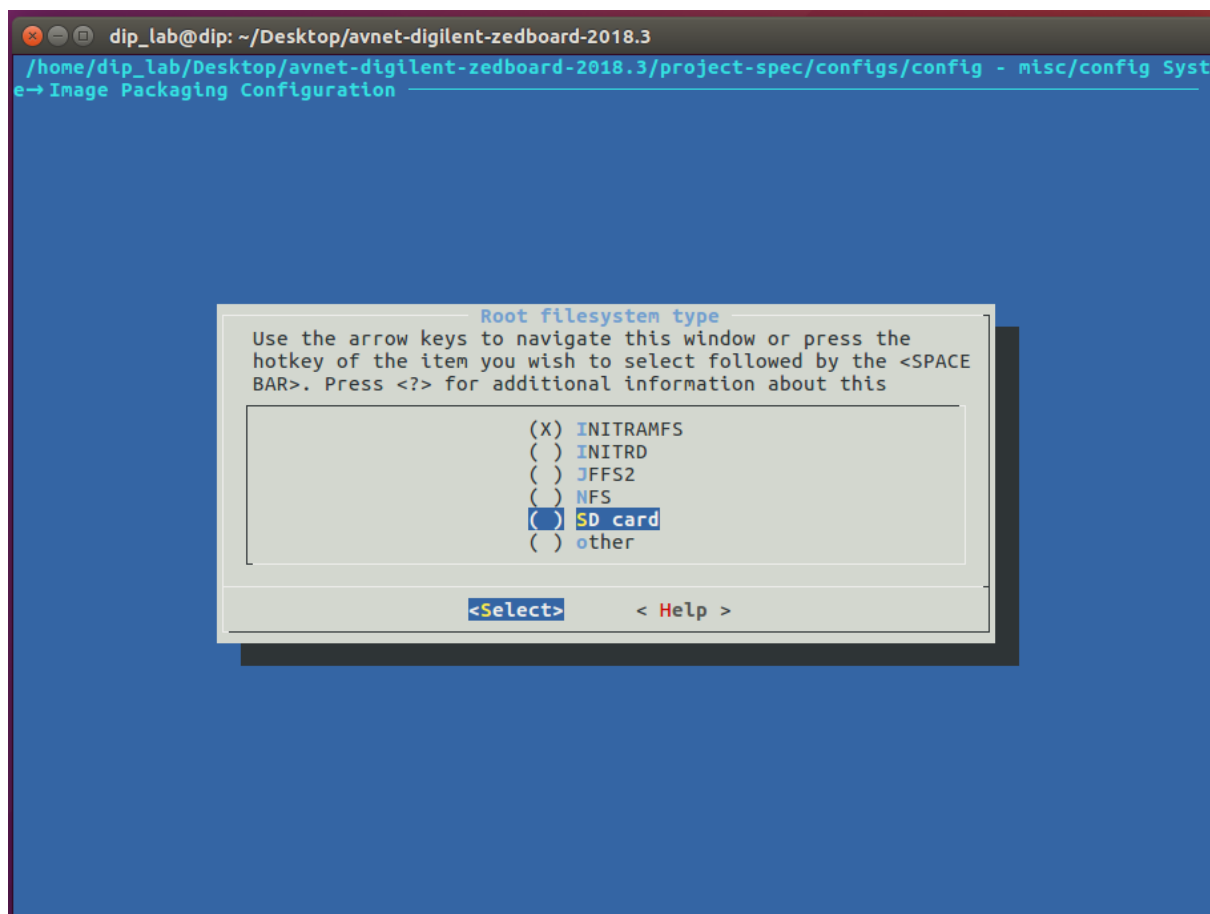
<Select>  < Exit >  < Help >  < Save >  < Load >
```

The screenshot shows a terminal window with the 'Image Packaging Configuration' menu. The menu is titled 'Image Packaging Configuration' and provides instructions on how to navigate using arrow keys, enter, and other shortcuts. The current selection is 'Root filesystem type (INITRAMFS)'. Below this, there are several options: '(image.ub) name for bootable kernel image', '(0x1000) DTB padding size', '[*] Copy final images to tftpboot', and '(/tftpboot) tftpboot directory'. At the bottom of the terminal, there is a status bar with the following options: '<Select>', '< Exit >', '< Help >', '< Save >', and '< Load >'.

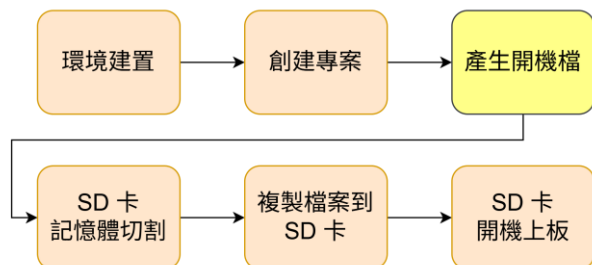
產生開機檔



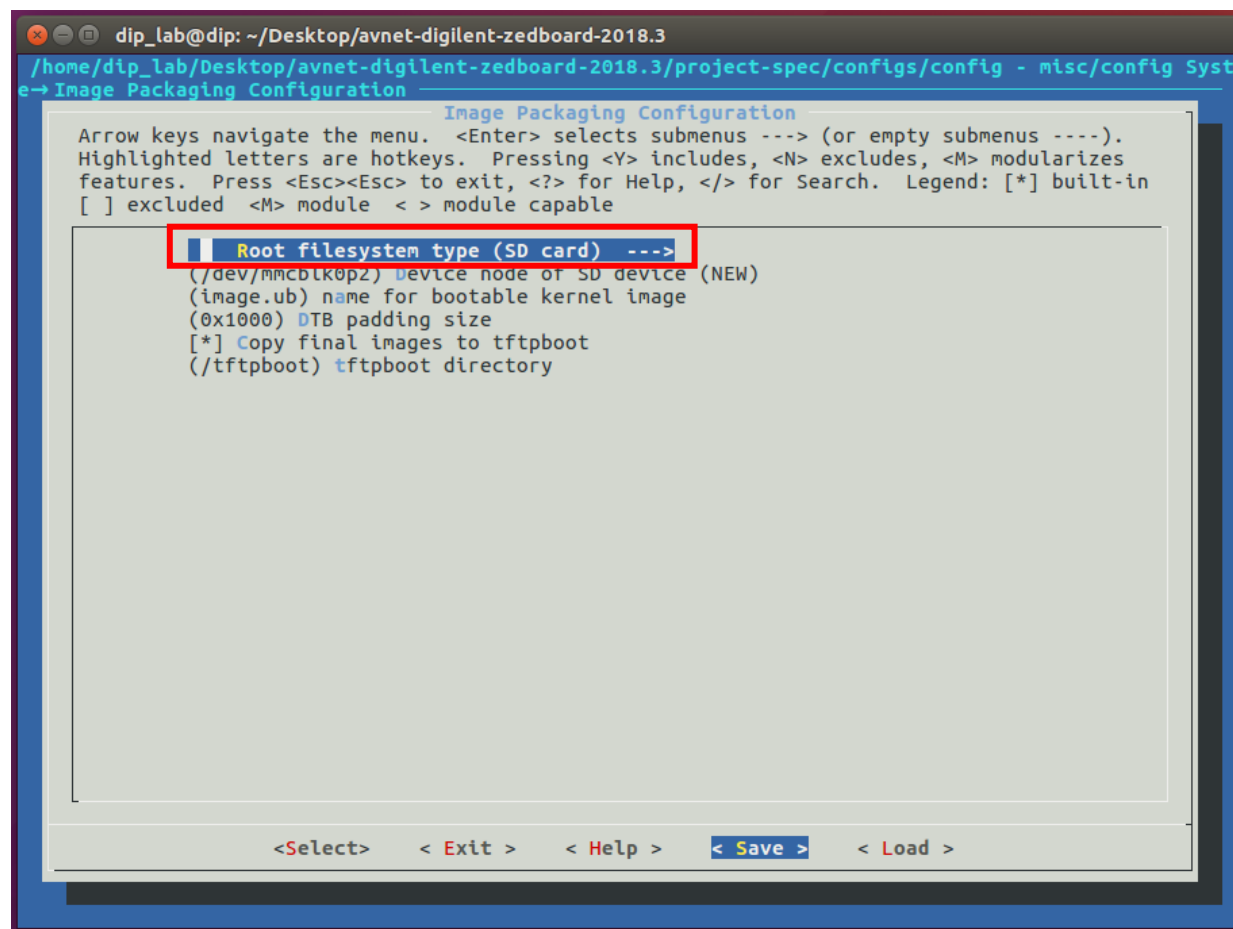
- 選" SD card"
- 按enter



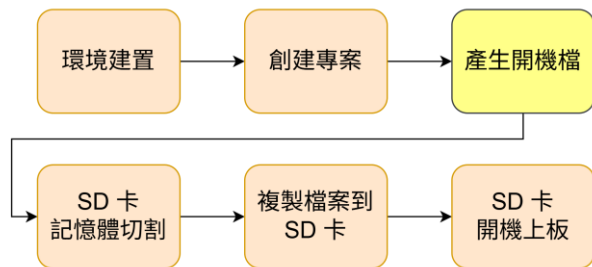
產生開機檔



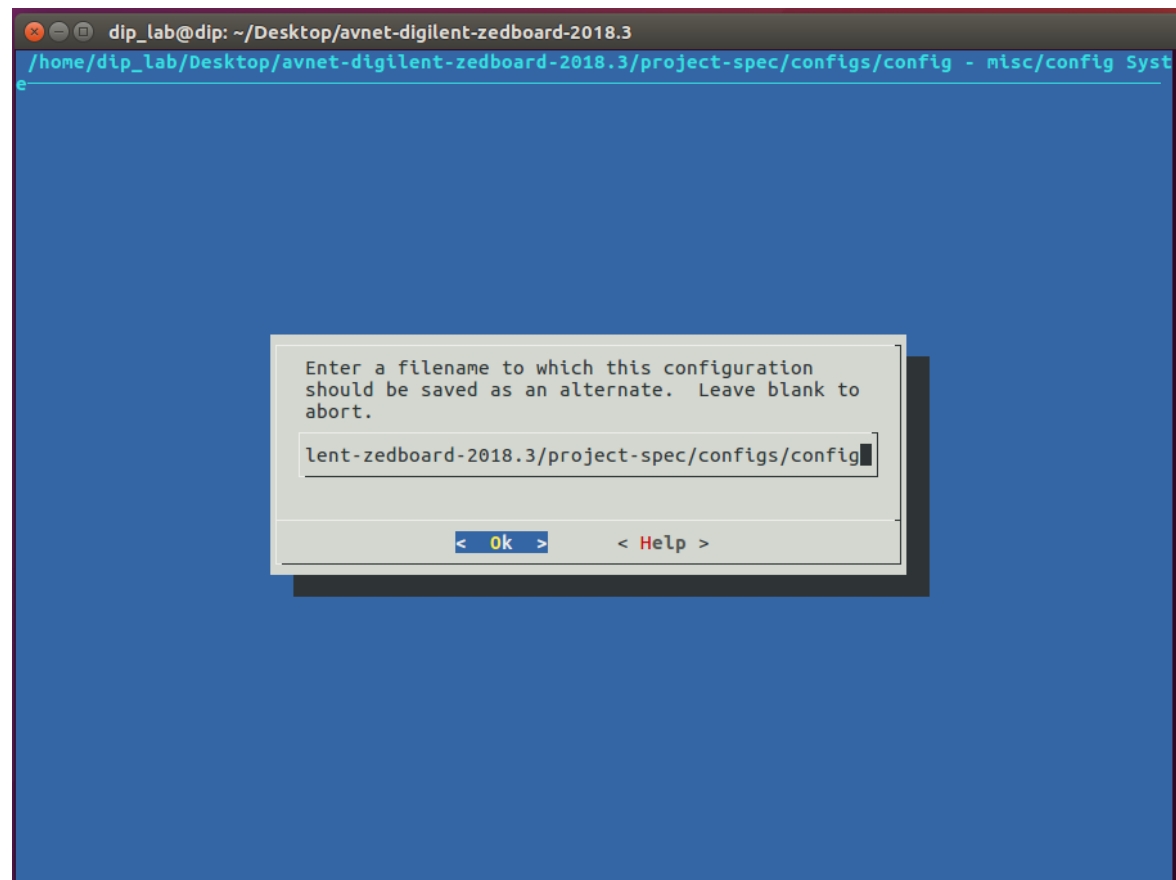
- 紅框處已改成你選的SD card
- 方向鍵，右鍵選到“ save”
- 按enter



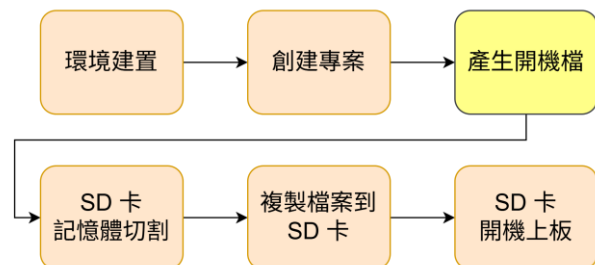
產生開機檔



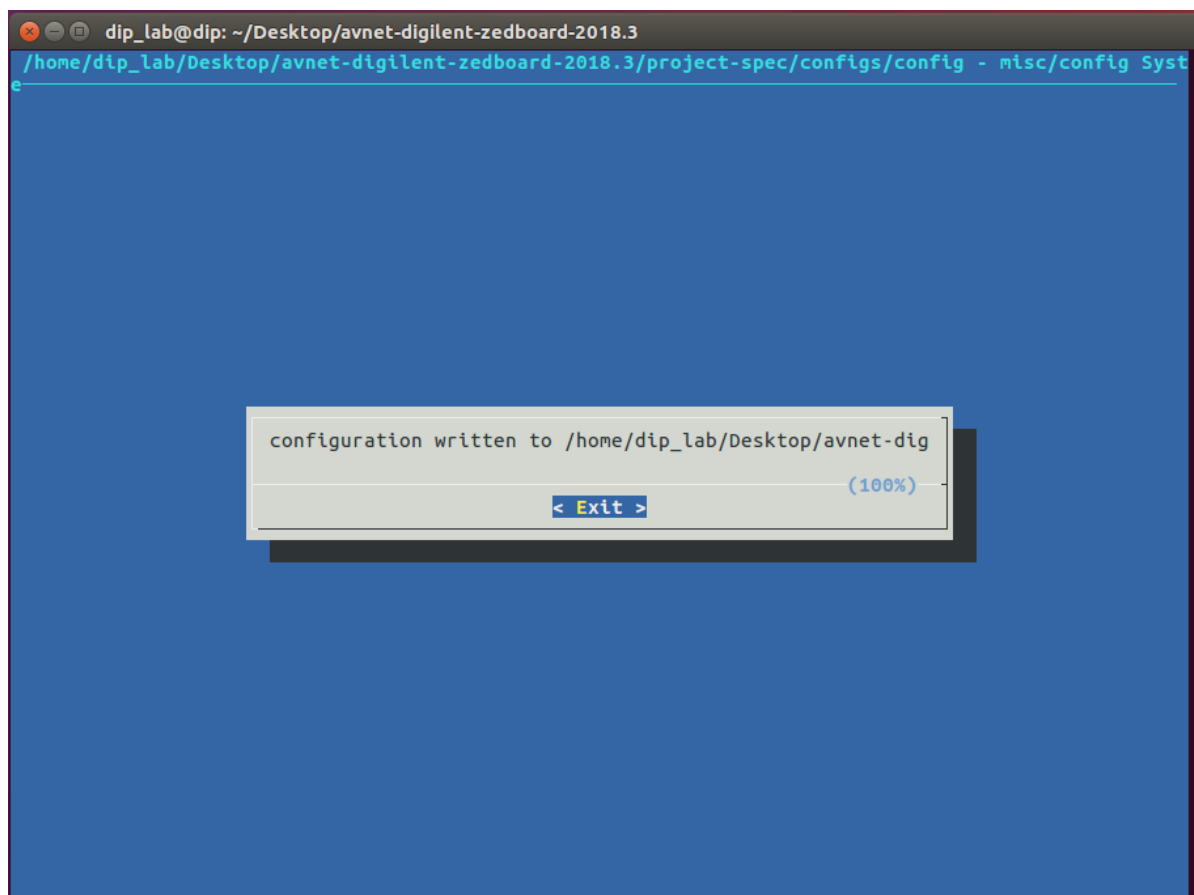
- 選“ OK”，按enter



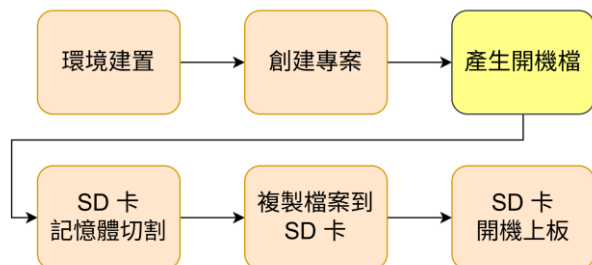
產生開機檔



- 按enter



產生開機檔



- 方向鍵，右鍵選“Exit”
- 按enter
- 接下來會等它跑一下，需要一點時間

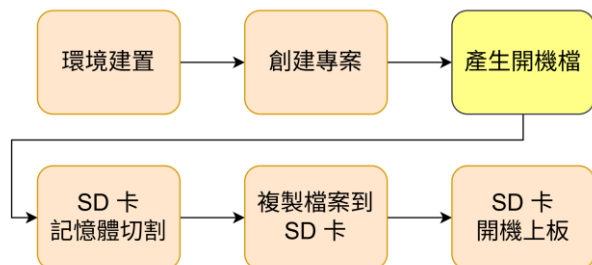
```
dip_lab@dip: ~/Desktop/avnet-digilent-zedboard-2018.3
/home/dip_lab/Desktop/avnet-digilent-zedboard-2018.3/project-spec/configs/config - misc/config Syst

misc/config System Configuration
Arrow keys navigate the menu. <Enter> selects submenus ---> (or empty submenus ----).
Highlighted letters are hotkeys. Pressing <Y> includes, <N> excludes, <M> modularizes
features. Press <Esc><Esc> to exit, <?> for Help, </> for Search. Legend: [*] built-in
[ ] excluded <M> module < > module capable

-- ZYNQ Configuration
Linux Components Selection --->
Auto Config Settings --->
-- Subsystem AUTO Hardware Settings --->
DTG Settings --->
u-boot Configuration --->
Image Packaging Configuration --->
Firmware Version Configuration --->
Yocto Settings --->

<Select>  <Exit >  < Help >  < Save >  < Load >
```

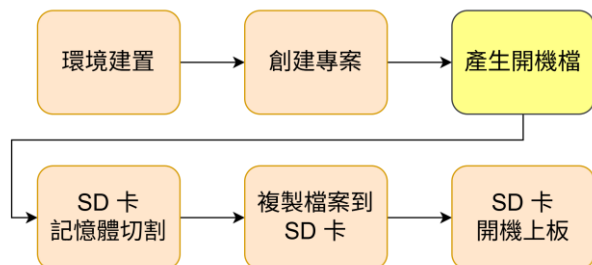
產生開機檔



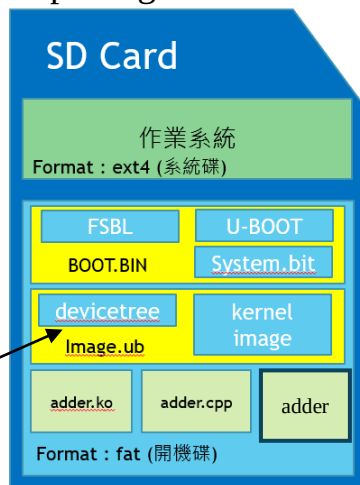
- 輸入指令：
- petalinux-build
- 這個步驟會稍久，它正在建置開機檔的相關檔案

```
ta@ubuntu: ~/Desktop/avnet-digilent-zedboard-2018.3
[INFO] oldconfig rootfs
[INFO] generating petalinux-user-image.bb
ta@ubuntu:~/Desktop/avnet-digilent-zedboard-2018.3$ petalinux-build
[INFO] building project
[INFO] sourcing bitbake
INFO: bitbake petalinux-user-image
Loading cache: 100% |#####| Time: 0:00:00
Loaded 3444 entries from dependency cache.
Parsing recipes: 100% |#####| Time: 0:00:03
Parsing of 2569 .bb files complete (2532 cached, 37 parsed). 3445 targets, 149 s
kipped, 0 masked, 0 errors.
NOTE: Resolving any missing task queue dependencies
Initialising tasks: 100% |#####| Time: 0:00:06
Checking sstate mirror object availability: 100% |#####| Time: 0:00:13
NOTE: Executing SetScene Tasks
NOTE: Executing RunQueue Tasks
Currently 1 running tasks (3038 of 3082) 98% |#####|
0: linux-xlnx-4.14-xilinx-v2018.3+gitAUTOINC+eeab73d120-r0 do_compile - 76s (pid
34297)
```

產生開機檔

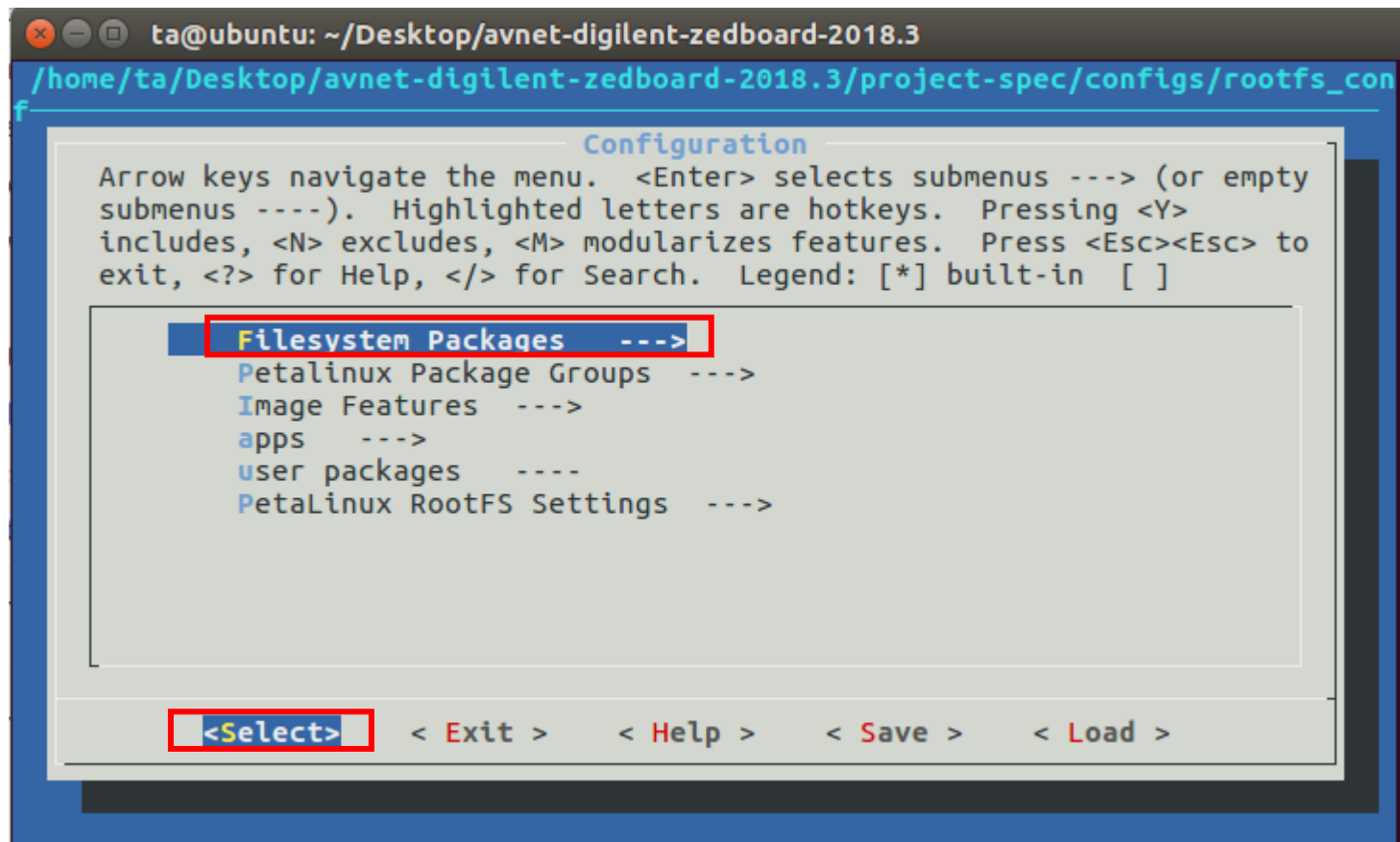


- 如果想要在建置開機檔時就下載一些常見的libraries到file systems中，輸入指令：
- `petalinux-config -c rootfs`
- 選filesystem packages

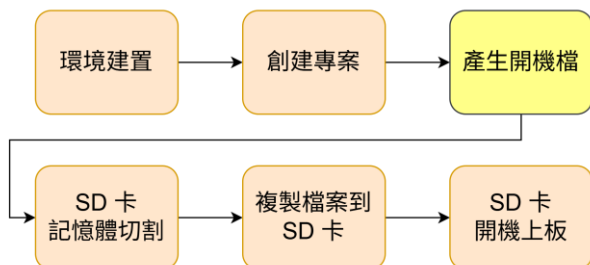


File system

```
ta@ubuntu:~/Desktop/avnet-digilent-zedboard-2018.3$ petalinux-config -c rootfs
```



產生開機檔



- 許多常見的library都在misc裡面
- Select 要安裝的函示庫(在比較下面)進入頁面，在要安裝的那欄按y，之後save
- 按y -> save，之後只要一直exit直到回到terminal page

輸入指令：

petalinux-build

- 因為會更新一些config檔，需要跑一下。

```
ta@ubuntu: ~/Desktop/avnet-digilent-zedboard-2018.3
/home/ta/Desktop/avnet-digilent-zedboard-2018.3/project-spec/configs/rootfs_conf
f→ Filesystem Packages

Filesystem Packages
Arrow keys navigate the menu. <Enter> selects submenus --- (or empty
submenus ----). Highlighted letters are hotkeys. Pressing <Y>
includes, <N> excludes, <M> modularizes features. Press <Esc><Esc> to
exit, <?> for Help, </> for Search. Legend: [*] built-in [ ]

admin --->
base --->
baseutils --->
console --->
devel --->
fonts --->
kernel --->
libs --->
misc --->
multimedia --->

Select < Exit >
```

```
ta@ubuntu: ~/Desktop/avnet-digilent-zedboard-2018.3
/home/ta/Desktop/avnet-digilent-zedboard-2018.3/project-spec/configs/rootfs_conf
f→ Filesystem Packages → misc

misc
Arrow keys navigate the menu. <Enter> selects submenus --- (or empty
submenus ----). Highlighted letters are hotkeys. Pressing <Y>
includes, <N> excludes, <M> modularizes features. Press <Esc><Esc> to
exit, <?> for Help, </> for Search. Legend: [*] built-in [ ]

openamp-fw-rpc-demo --->
opkg-arch-config --->
orc --->
p11-kit --->
packagegroup-core-boot --->
packagegroup-core-buildessential --->
packagegroup-core-sdk --->
packagegroup-core-ssh-dropbear --->
packagegroup-core-standalone-sdk-target --->
packagegroup-core-tools-debug --->

Select < Exit >
```

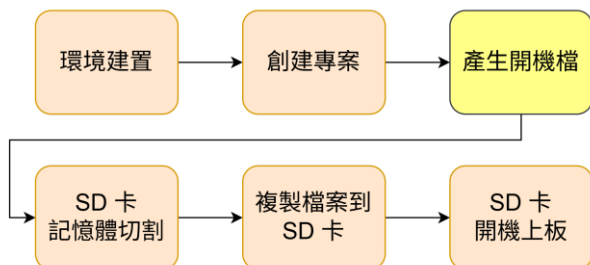
```
ta@ubuntu: ~/Desktop/avnet-digilent-zedboard-2018.3
/home/ta/Desktop/avnet-digilent-zedboard-2018.3/project-spec/configs/rootfs_conf
f→ Filesystem Packages → misc → packagegroup-core-buildessential

packagegroup-core-buildessential
Arrow keys navigate the menu. <Enter> selects submenus --- (or empty
submenus ----). Highlighted letters are hotkeys. Pressing <Y>
includes, <N> excludes, <M> modularizes features. Press <Esc><Esc> to
exit, <?> for Help, </> for Search. Legend: [*] built-in [ ]

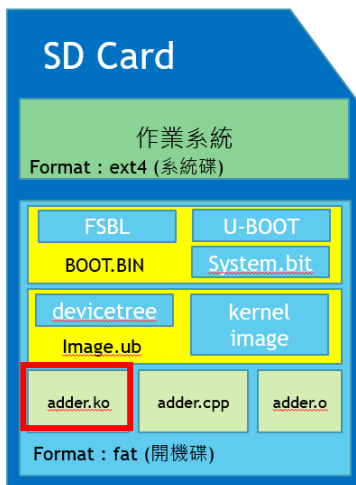
[*] packagegroup-core-buildessential
[ ] packagegroup-core-buildessential-dev
[ ] packagegroup-core-buildessential-dbg

< Select > < Exit > < Help > Save < Load >
```

產生開機檔



- 輸入指令：
- `petalinux-create -t modules -n adder --enable`
- Adder : 專案名
- 產生一個driver模板(adder.ko)，這個模板是可以根據需求更改的



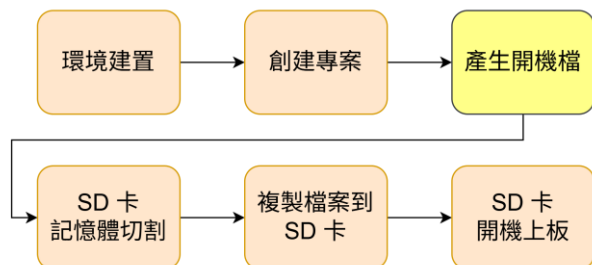
```
ta@ubuntu: ~/Desktop/avnet-digilent-zedboard-2018.3
$ petalinux-create -t project -n <PROJECT> --template zynqMP

Create an app and enable it:
$ petalinux-create -t apps -n myapp --enable
The application "myapp" will be created with c template in:
<PROJECT>/project-spec/meta-user/recipes-apps/myapp

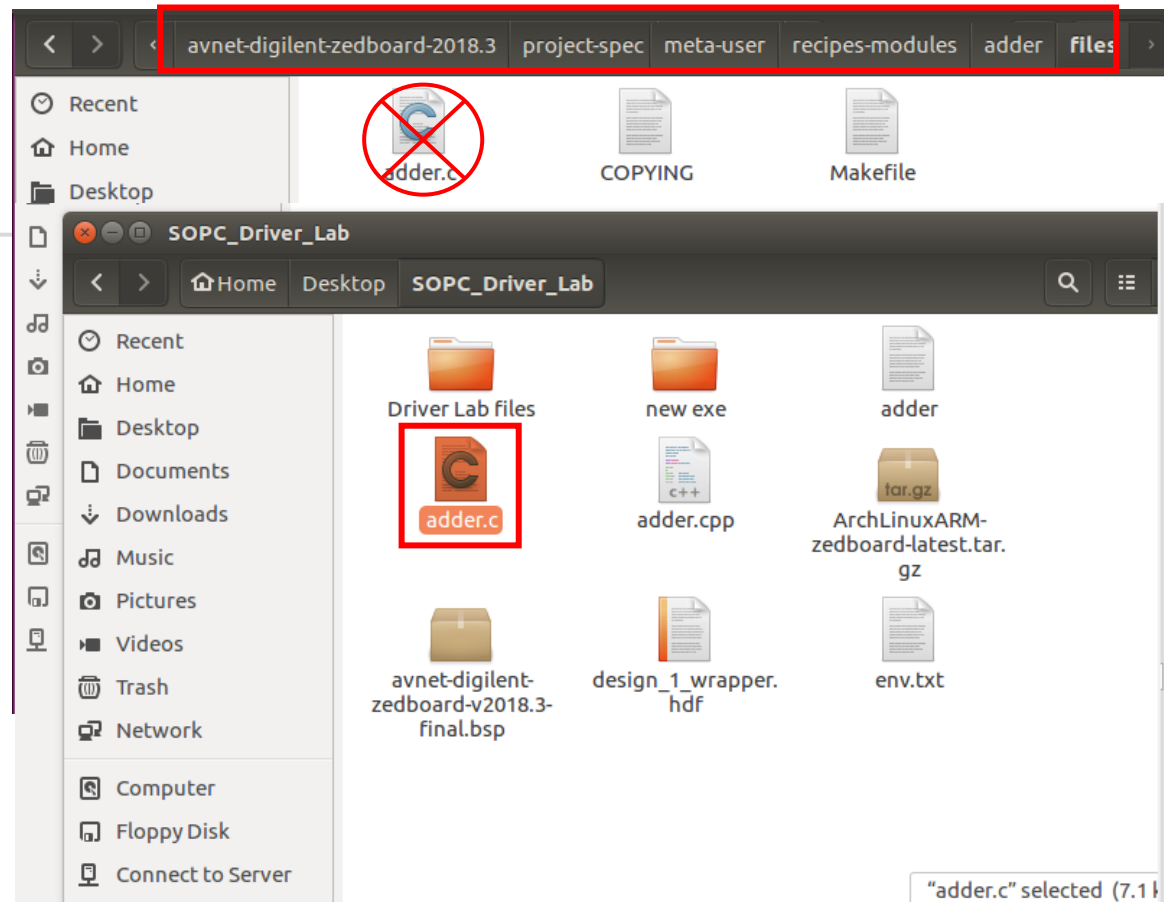
Create an module and enable it:
$ petalinux-create -t modules -n mymodule --enable
The module "mymodule" will be created with template in:
<PROJECT>/project-spec/meta-user/recipes-modules/mymodule

ta@ubuntu:~/Desktop/avnet-digilent-zedboard-2018.3$ petalinux-create -t modules
-n adder --enable
INFO: Create modules: adder
INFO: New modules successfully created in /home/ta/Desktop/avnet-digilent-zedboa
rd-2018.3/project-spec/meta-user/recipes-modules/adder
INFO: Enabling created component...
INFO: sourcing bitbake
INFO: oldconfig rootfs
INFO: adder has been enabled
ta@ubuntu:~/Desktop/avnet-digilent-zedboard-2018.3$
```

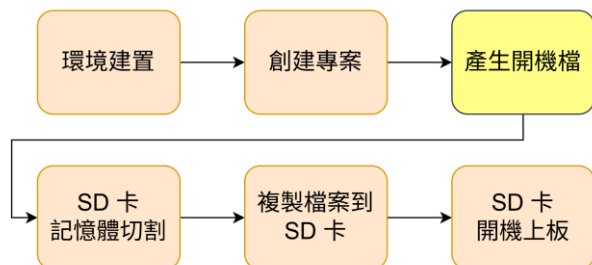

產生開機檔



- 點開 avnet-digilent-zedboard-2018.3 資料夾看框框路徑找到建立的 module，另開 SOPC_SOPC_Driver_Lab
- 將以下路徑資料夾裡的 adder.c 刪除，並把提前給同學的 SOPC_Driver_Lab/adder.c 覆蓋到 avnet-digilent-zedboard-2018.3 資料夾中



產生開機檔



- 請同學點開複製過去的adder.c，裡面compatible的值需要自行更改，要去哪裡找compatible的值請看下頁PPT
- 由於須根據device tree中的硬體描述訊息，確定該driver匹配的硬體的名稱。

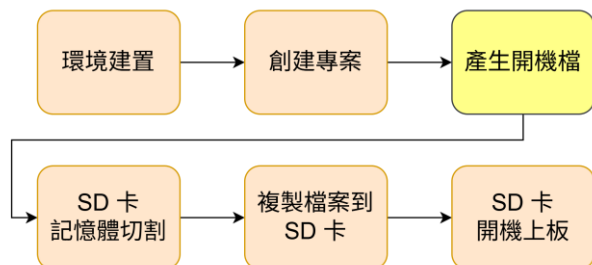
```
Open [icon] Save

static int adder_remove(struct platform_device *pdev)
{
    struct device *dev = &pdev->dev;
    struct adder_local *lp = dev_get_drvdata(dev);
    cdevice_exit(lp);
    iounmap(lp->base_addr);
    release_mem_region(lp->mem_start, lp->mem_end - lp->mem_start + 1);
    kfree(lp);
    dev_set_drvdata(dev, NULL);
    return 0;
}

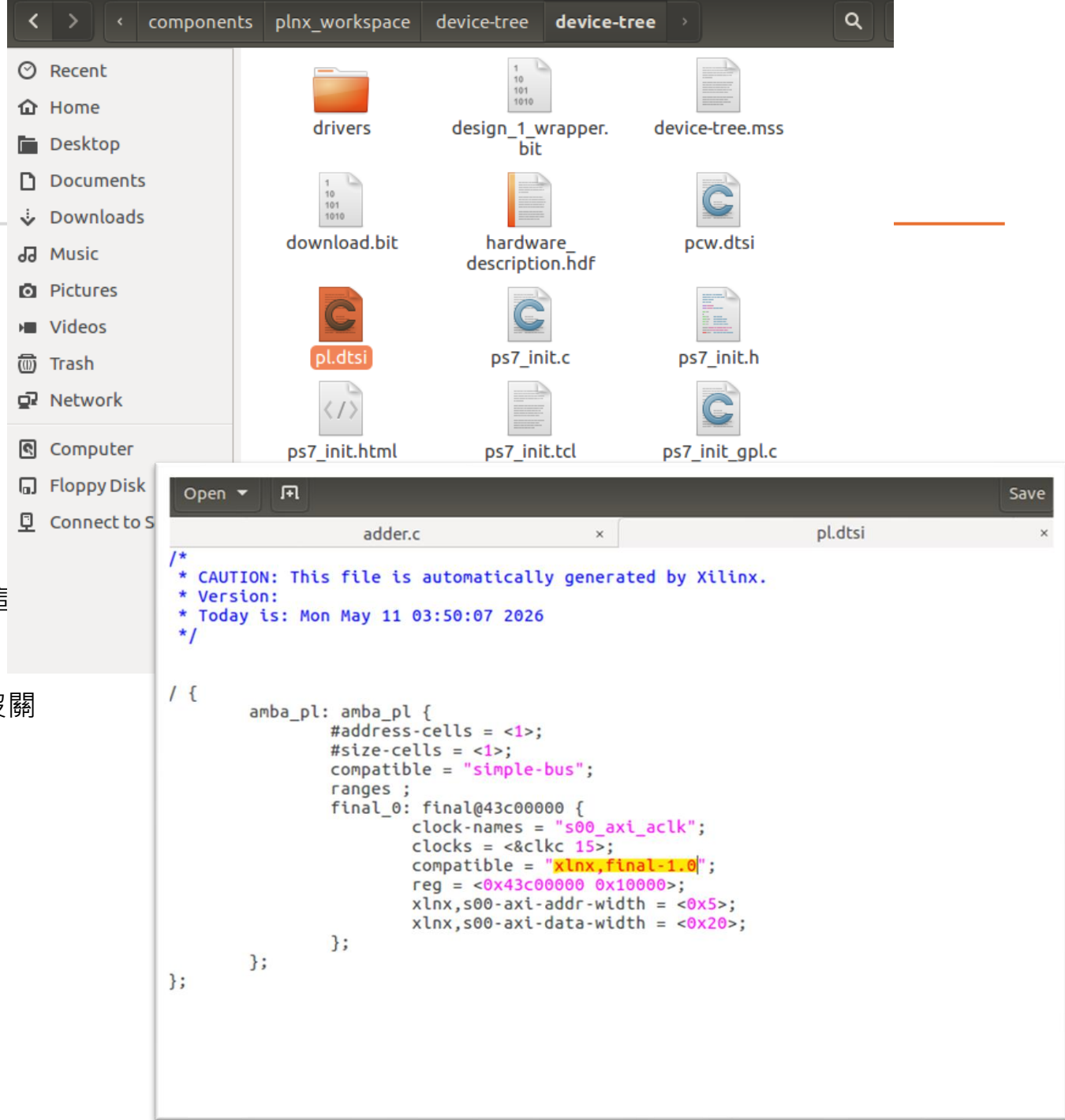
//compatible要查了才知道
#ifdef CONFIG_OF
static struct of_device_id adder_of_match[] = {
    { .compatible = "xlnx,final-1.0", },
    { /* end of list */ },
};
MODULE_DEVICE_TABLE(of, adder_of_match);
#else
#define adder_of_match
#endif

static struct platform_driver adder_driver = {
    .driver = {
        .name = DRIVER_NAME,
        .owner = THIS_MODULE,
        .of_match_table = adder_of_match,
    },
    .probe = adder_probe,
    .remove = adder_remove,
};
```

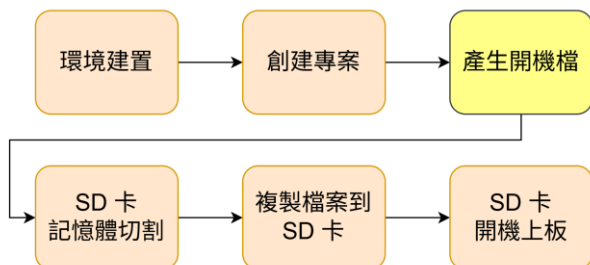
產生開機檔



- 請到~Desktop/avnet-digilent-zedboard-v2018.3/component/plnx_workspace/device-tree/device-tree/ 這路徑下點開pl.dtsi
- 複製"xlnx,final-1.0"，此即你能用的compatible值
- 注意：你們看到的compatible值可能跟我不一樣，不一樣沒關係，一樣複製下來



產生開機檔



- adder.c中如右圖紅框處，將剛剛複製的，取代原本的compatible值，並save

```
File Edit View Search Tools Documents Help 10:54 PM
Open [icon] Save
static int adder_remove(struct platform_device *pdev)
{
    struct device *dev = &pdev->dev;
    struct adder_local *lp = dev_get_drvdata(dev);
    cdevice_exit(lp);
    iounmap(lp->base_addr);
    release_mem_region(lp->mem_start, lp->mem_end - lp->mem_start + 1);
    kfree(lp);
    dev_set_drvdata(dev, NULL);
    return 0;
}

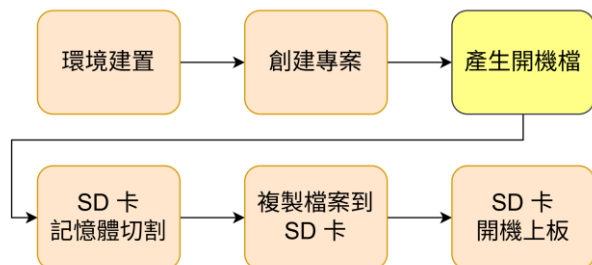
//compatible要查了才知道
#ifdef CONFIG_OF
static struct of_device_id adder_of_match[] = {
    { .compatible = "xlnx,final-1.0", },
    { /* end of list */ },
};
MODULE_DEVICE_TABLE(of, adder_of_match);
#else
# define adder_of_match
#endif

static struct platform_driver adder_driver = {
    .driver = {
        .name = DRIVER_NAME,
        .owner = THIS_MODULE,
        .of_match_table = adder_of_match,
    },
    .probe = adder_probe,
    .remove = adder_remove,
};

static int __init adder_init(void)
{
```

The screenshot shows a code editor with a menu bar (File, Edit, View, Search, Tools, Documents, Help) and a status bar (10:54 PM). The code is in C and defines a platform driver. A red box highlights the line `{ .compatible = "xlnx,final-1.0", },` in the `adder_of_match` array. A blue box highlights the `Save` button in the top right corner of the editor window.

產生開機檔

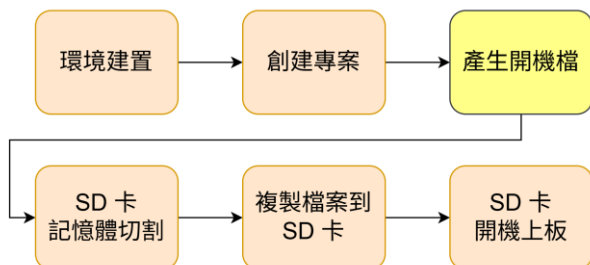


- 將adder.c編譯成adder.ko，輸入指令：
petalinux-build -c adder

```
ta@ubuntu: ~/Desktop/avnet-digilent-zedboard-2018.3
$ petalinux-create -t modules -n mymodule --enable
The module "mymodule" will be created with template in:
<PROJECT>/project-spec/meta-user/recipes-modules/mymodule

ta@ubuntu:~/Desktop/avnet-digilent-zedboard-2018.3$ petalinux-create -t modules
-n adder --enable
INFO: Create modules: adder
INFO: New modules successfully created in /home/ta/Desktop/avnet-digilent-zedboa
rd-2018.3/project-spec/meta-user/recipes-modules/adder
INFO: Enabling created component...
INFO: sourcing bitbake
INFO: oldconfig rootfs
INFO: adder has been enabled
ta@ubuntu:~/Desktop/avnet-digilent-zedboard-2018.3$ petalinux-build -c adder
[INFO] building adder
[INFO] sourcing bitbake
INFO: bitbake adder
Loading cache: 100% |#####| Time: 0:00:00
Loaded 3444 entries from dependency cache.
Parsing recipes: 100% |#####| Time: 0:00:03
Parsing of 2570 .bb files complete (2536 cached, 34 parsed). 3446 targets, 149 s
kipped, 0 masked, 0 errors.
NOTE: Resolving any missing task queue dependencies
Initialising tasks: 82% |#####| ETA: 0:00:00
```

產生開機檔



- 此lab已經先有編譯好的檔案了，此步驟同學自行參考

- adder.cpp的兩種編譯(編譯出.o => link libraries execute file)方式：

(1)在虛擬機底下編譯：

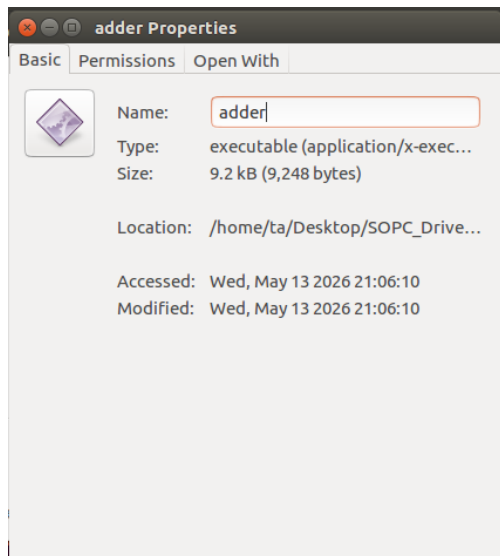
- `source ~/build/settings.sh`
- `arm-linux-gnueabihf-g++ adder.cpp -o adder`
- `arm-linux-gnueabihf-g++ -fPIE -pie adder.cpp -o adder`
- 以上方法不行可參考：

<https://www.linuxcool.com/%E5%A6%82%E4%BD%95%E5%9C%A864%E4%BD%8Dubuntu%E7%B3%BB%E7%BB%9F%E4%B8%8A%E5%AE%89%E8%A3%8564%E4%BD%8Darm-linux-gcc%E4%BA%A4%E5%8F%89%E7%BC%96%E8%AF%91%E5%99%A8>

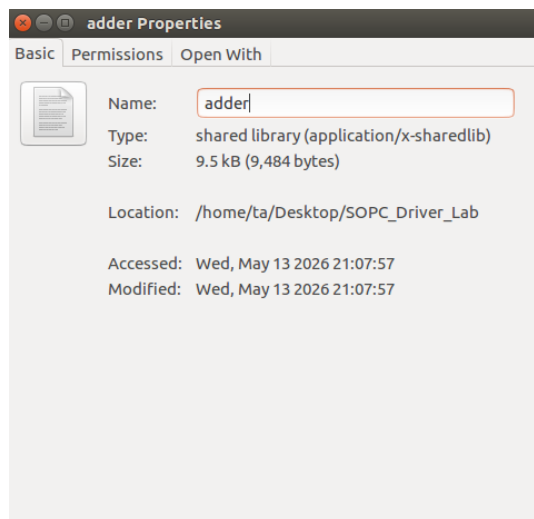
(2)在FPGA板編譯：(要接網路線) 不建議，會遇到版本不符合問題

- `pacman-key --init`
- `pacman-key --populate archlinuxarm`
- `pacman -Syu`
- `pacman -S base-devel (選10(gcc))`
- `arm-linux-gnueabihf-g++ adder.cpp -o adder`
- `arm-linux-gnueabihf-g++ -fPIE -pie adder.cpp -o adder`

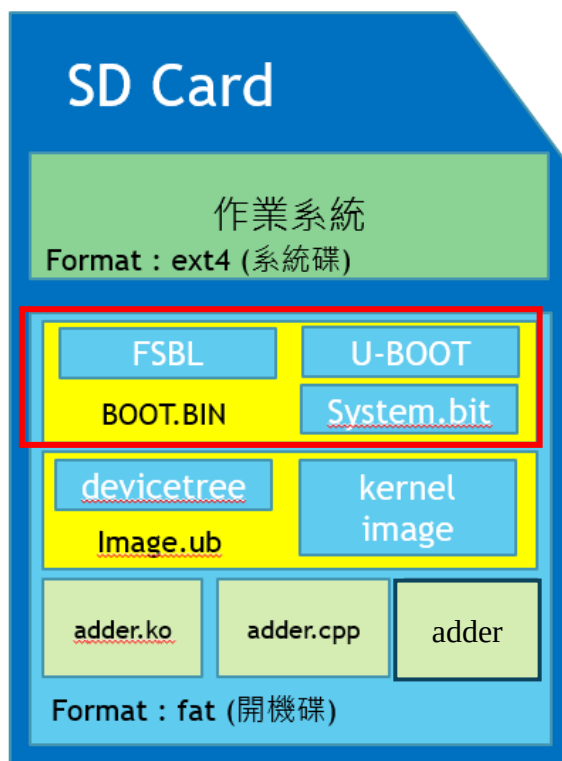
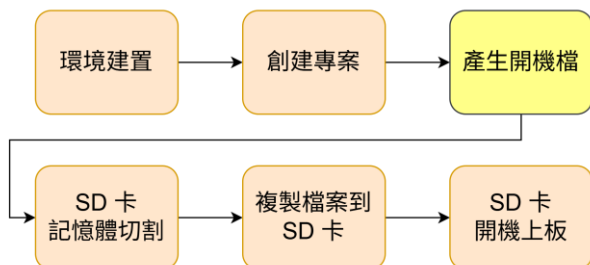
adder.o



adder.exe



產生開機檔



- 輸入指令：產生BOOT.bin
- `petalinux-package --boot --fpga images/linux/system.bit --fsbl images/linux/zynq_fsbl.elf --u-boot --force`

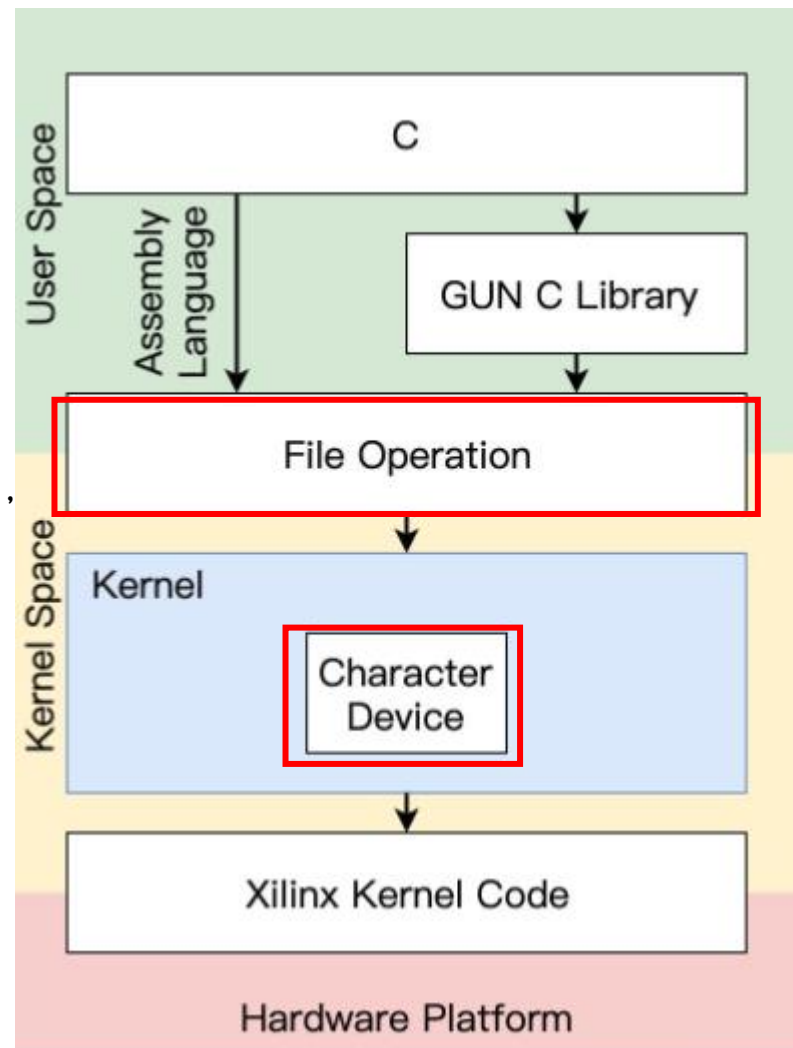
```
ta@ubuntu: ~/Desktop/avnet-digilent-zedboard-2018.3
NOTE: Tasks Summary: Attempted 736 tasks of which 719 didn't need to be rerun and all succeeded.
INFO: Copying Images from deploy to images
NOTE: Failed to copy built images to tftp dir: /tftpboot
[INFO] successfully built adder
ta@ubuntu:~/Desktop/avnet-digilent-zedboard-2018.3$ petalinux-package --boot --fpga images/linux/system.bit --fsbl images/linux/zynq_fsbl.elf --u-boot --force
INFO: File in BOOT BIN: "/home/ta/Desktop/avnet-digilent-zedboard-2018.3/images/linux/zynq_fsbl.elf"
INFO: File in BOOT BIN: "/home/ta/Desktop/avnet-digilent-zedboard-2018.3/images/linux/system.bit"
INFO: File in BOOT BIN: "/home/ta/Desktop/avnet-digilent-zedboard-2018.3/images/linux/u-boot.elf"
INFO: Generating Zynq binary package BOOT.BIN...

***** Xilinx Bootgen v2018.3
**** Build date : Nov 15 2018-19:22:29
** Copyright 1986-2018 Xilinx, Inc. All Rights Reserved.

INFO: Binary is ready.
WARNING: Unable to access the TFTPBOOT folder /tftpboot!!!
WARNING: Skip file copy to TFTPBOOT folder!!!
ta@ubuntu:~/Desktop/avnet-digilent-zedboard-2018.3$
```


如何修改Driver(補充)

- ◆ 原始 template 僅提供 Platform Driver 架構，缺少 user space 溝通接口，所以必須加入 **File Operation & Character Device** 來呼叫 **Kernel** 操作底層的硬體。
1. 在 `adder_local` 結構，加入你所需要的設備資訊(kernel-char device)新增你需要的使用者呼叫驅動函式的結構(file_operations)，讓應用程式能呼叫驅動去做你需要的動作
 2. 建立 char device 初始化與清除函式：`cdevice_init()` 與 `cdevice_exit()`，負責註冊/刪除字元裝置，創建節點等。
 3. 整合到 Platform Driver：在 `probe()` 中呼叫 `cdevice_init()` 完成初始化；在 `remove()` 中呼叫 `cdevice_exit()` 做資源釋放或註銷



如何修改Driver(補充)

1. 在 adder_local 結構，加入你所需要的設備資訊(kernel -char device)

◆ template

```
struct adder_local {  
    int irq;  
    unsigned long mem_start;  
    unsigned long mem_end;  
    void __iomem *base_addr;  
    struct device *__iomem *base_addr  
};|
```

◆ adder.c

```
Open ▾   
  
#define DRIVER_NAME "adder-master"  
  
#define input_a      0  
#define input_b      1  
#define output_a     3  
#define output_b     4  
#define output_c     5  
#define output_d     6  
  
struct adder_local {  
    unsigned long mem_start;  
    unsigned long mem_end;  
    void __iomem *base_addr;  
    struct device *plat_dev_p; //Pointer of Platform Device  
  
    //Char Device  
    struct device *char_dev_p; //Pointer of Char Device  
    dev_t dev_node;  
    struct cdev cdev;  
    struct class *class_p;  
};
```

如何修改Driver(補充)

1. 在 adder_local 結構，加入你所需要的設備資訊(kernel –file operation)

◆ adder.c

```
//此為driver中的一項重要功能，透過它來傳值，不論是讀還是寫
static long ioctl(struct file *file_p, unsigned int cmd, unsigned long arg)
{
    struct adder_local *lp = (struct adder_local *)file_p->private_data;
    int __iomem *tmp = lp->base_addr;
    if(cmd>7){
        cmd -= 64*1024;
    }
    switch(cmd){
        case input_a:
            tmp[0] = arg;
            printk("the value is %d\n", tmp[0]);
            printk("\n");
            break;
        case input_b:
            tmp[1] = arg;
            printk("the value is %d\n", tmp[1]);
            printk("\n");
            break;
        case output_a:
            printk("in output_a\n");
            return tmp[3];
        case output_b:
            printk("in output_b\n");
            return tmp[4];
        case output_c:
            printk("in output_c\n");
            return tmp[5];
        case output_d:
            printk("in output_d\n");
            return tmp[6];
    }
    return 0;
}
```

```
static int open(struct inode *ino, struct file *file_p)
{
    file_p->private_data = container_of(ino->i_cdev, struct adder_local, cdev);
    return 0;
}

static struct file_operations dm_fops = {
    .owner      = THIS_MODULE,
    .open       = open,
    .unlocked_ioctl = ioctl,
};
```

如何修改Driver(補充)

2. 建立 char device 初始化與清除函式：cdevice_init() 與 cdevice_exit()，負責註冊/刪除字元裝置，創建節點等。

◆ adder.c

```
/*
 * Initial Char Device Enable
 */
static int cdevice_init(struct adder_local *lp)
{
    int rc;
    static struct class *local_class_p = NULL;
    /*
     * Allocate a character device from the kernel for this driver.
     */
    rc = alloc_chrdev_region(&lp->dev_node, 0, 1, DRIVER_NAME);
    if (rc) {
        dev_err(lp->plat_dev_p, "unable to get a char device number\n");
        return rc;
    }
    /*
     * Initialize the device data structure before registering the character
     * device with the kernel.
     */
    cdev_init(&lp->cdev, &dm_fops);
    lp->cdev.owner = THIS_MODULE;

    rc = cdev_add(&lp->cdev, lp->dev_node, 1);
    if (rc) {
        dev_err(lp->plat_dev_p, "unable to add char device\n");
        goto init_error1;
    }
    /*
     * Only one class in sysfs is to be created for multiple channels,
     * create the device in sysfs which will allow the device node
     * in /dev to be created
     */
}
```

```
/*-----
 * Free Char Device
 *-----
static void cdevice_exit(struct adder_local *lp)
{
    if (lp->char_dev_p) {
        device_destroy(lp->class_p, lp->dev_node);
        class_destroy(lp->class_p);
        cdev_del(&lp->cdev);
        unregister_chrdev_region(lp->dev_node, 1);
    }
}
```

如何修改Driver(補充)

2. 建立 char device 初始化與清除函式：cdevice_init() 與 cdevice_exit()，負責註冊/刪除字元裝置，創建節點等，剛才的 cdevice_init 中加**建立節點邏輯**，目的是要讓這個驅動程式能夠被用戶空間以 /dev/adder-master 為裝置檔案來存取，以及IS_ERR與error偵錯邏輯。

◆ adder.c

```
        return rc;
    }
    /*
     * Initialize the device data structure before registering the character
     * device with the kernel.
     */
    cdev_init(&lp->cdev, &dm_fops);
    lp->cdev.owner = THIS_MODULE;

    rc = cdev_add(&lp->cdev, lp->dev_node, 1);
    if (rc) {
        dev_err(lp->plat_dev_p, "unable to add char device\n");
        goto init_error1;
    }
    /*
     * Only one class in sysfs is to be created for multiple channels,
     * create the device in sysfs which will allow the device node
     * in /dev to be created
     */
    if (!local_class_p) {
        local_class_p = class_create(THIS_MODULE, DRIVER_NAME);
        if (IS_ERR(lp->plat_dev_p->class)) {
            dev_err(lp->plat_dev_p, "unable to create class\n");
            rc = -1;
            goto init_error2;
        }
    }
    lp->class_p = local_class_p;
    /*
     * Create the device node in /dev so the device is accessible
     * as a character device
     */
    lp->char_dev_p = device_create(lp->class_p, NULL,
                                lp->dev_node, NULL, DRIVER_NAME);
    if (IS_ERR(lp->plat_dev_p)) {
        dev_err(lp->plat_dev_p, "unable to create the char device\n");
        goto init_error3;
    }
    return 0;

init_error3:
    class_destroy(lp->class_p);

init_error2:
    cdev_del(&lp->cdev);
```

如何修改Driver(補充)

3. 整合到 Platform Driver：在 probe() 中呼叫 cdevice_init() 完成初始化；在 remove() 中呼叫 cdevice_exit() 做資源釋放或註銷，框框是要加上的東西

◆ adder.c

Platform Driver 與裝置樹 (Device Tree) 對應，用來與實際硬體裝置綁定 (compatible)，當硬體初始化會呼叫 `probe()`，卸載時會呼叫 `remove()`。

```
dev_info(dev, "adder at 0x%08x mapped to 0x%08x\n",
         (unsigned int __force)lp->mem_start,
         (unsigned int __force)lp->base_addr
        );
lp->plat_dev_p = &pdev->dev;
rc = cdevice_init(lp);
if(rc) {
    dev_err(dev, "Char device initial failed!\n");
    return -1;
}

return 0;

error2:
release_mem_region(lp->mem_start, lp->mem_end - lp->mem_start + 1);
error1:
kfree(lp);
dev_set_drvdata(dev, NULL);
return rc;
}
```

```
static int adder_remove(struct platform_device *pdev)
{
    struct device *dev = &pdev->dev;
    struct adder local *lp = dev_get_drvdata(dev);
    cdevice_exit(lp);
    iounmap(lp->base_addr);
    release_mem_region(lp->mem_start, lp->mem_end - lp->mem_start + 1);
    kfree(lp);
    dev_set_drvdata(dev, NULL);
    return 0;
}
```

如何修改Driver(補充)

2. adder.c & adder.cpp 之對應

◆ adder.c

```
#define DRIVER_NAME "adder-master"

#define input_a      0
#define input_b      1
#define output_a     3
#define output_b     4
#define output_c     5
#define output_d     6

static long ioctl(struct file *file_p, unsigned int cmd, unsigned long arg)
{
    struct adder_local *lp = (struct adder_local *)file_p->private_data;
    int iomem *tmp = lp->base_addr;
    if(cmd>7){
        cmd -= 64*1024;
    }

    switch(cmd){
        case input_a:
            tmp[0] = arg;
            printk("the value is %d\n", tmp[0]);
            printk("\n");
            break;
        case input_b:
            tmp[1] = arg;
            printk("the value is %d\n", tmp[1]);
            printk("\n");
            break;
        case output_a:
            printk("in output_a\n");
            return tmp[3];
        case output_b:
            printk("in output_b\n");
            return tmp[4];
        case output_c:
            printk("in output_c\n");
            return tmp[5];
        case output_d:
            printk("in output_d\n");
            return tmp[6];
    }

    return 0;
}
```

重要功能，透過它來傳值，不論是讀還是寫

◆ adder.cpp

```
#include <iomanip>
#include <sys/mman.h>
#include <sys/ioctl.h>
#include <unistd.h>
#include <fcntl.h>
#include <unistd.h>

int main(int argc, char ** argv){

    int test_fp;
    int *virt_addr;
    int phy_addr;

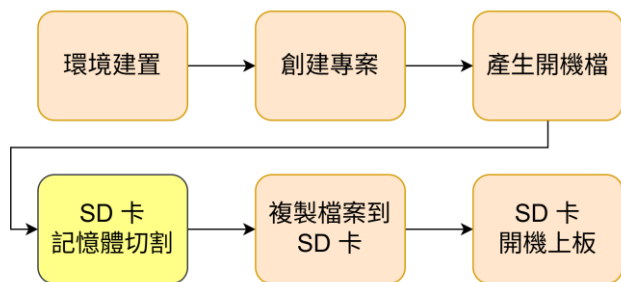
    /*
     * After insmod .ko (driver), you will be able to see test(DRIVER_NAME) come up,
     * Open it like file and receive a driver pointer;
     */
    test_fp = open("/dev/adder-master", O_RDWR);
    if(test_fp == 0){
        std::cout << "Can not open /dev/test" << std::endl;
        return -1;
    }

    ioctl(test_fp, 0+64*1024, 100 );
    ioctl(test_fp, 1+64*1024, 50 );

    sleep(1); // Wait For IP Finish
    std::cout << "add : " << ioctl(test_fp, 3,0) << std::endl;
    std::cout << "sub : " << ioctl(test_fp, 4,0) << std::endl;
    std::cout << "mul : " << ioctl(test_fp, 5,0) << std::endl;
    std::cout << "div : " << ioctl(test_fp, 6,0) << std::endl;

    return 0;
}
```

SD card – 記憶體切割

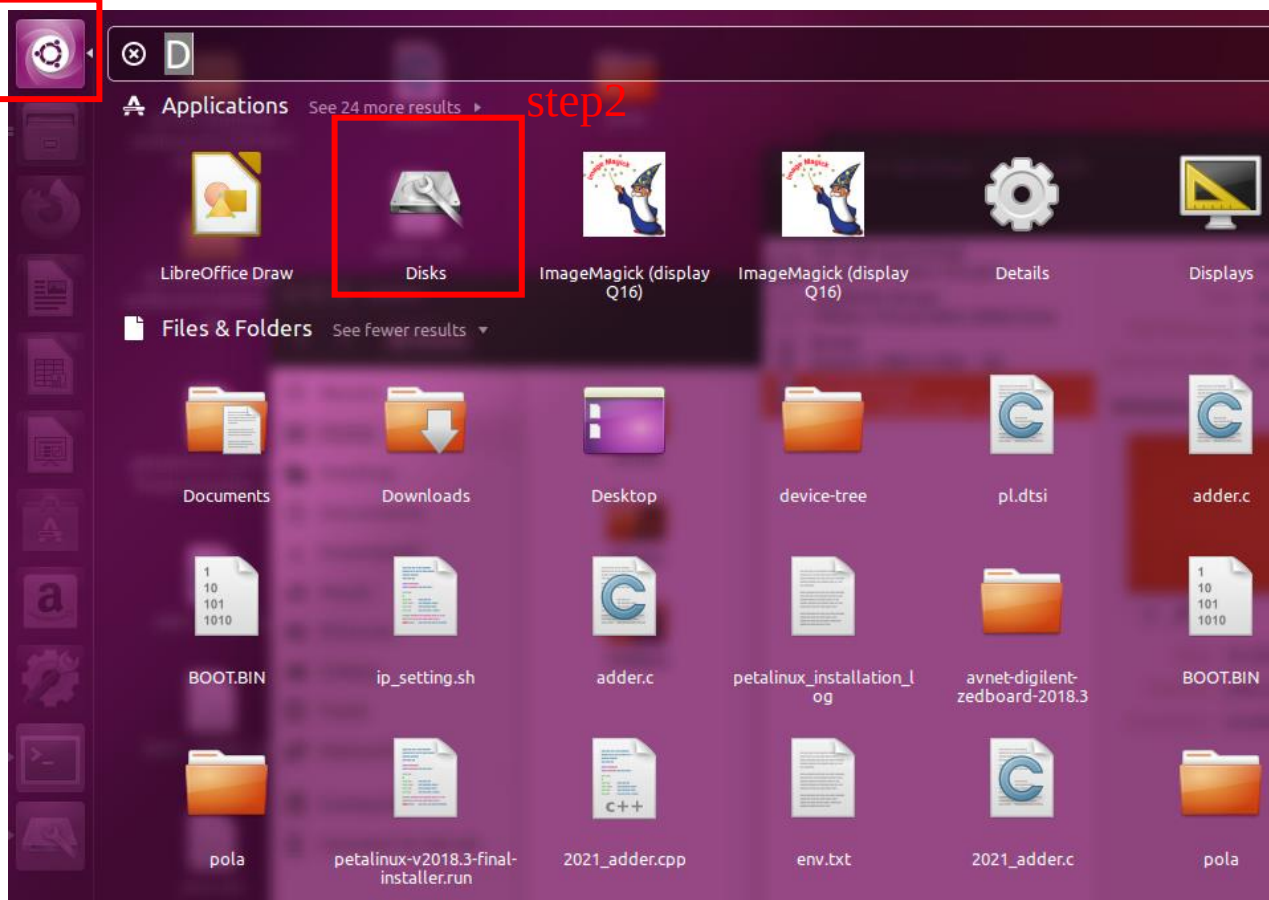


- 把SD卡插到轉接器，並將轉接器插到電腦
- 接著如右圖所示依序點開

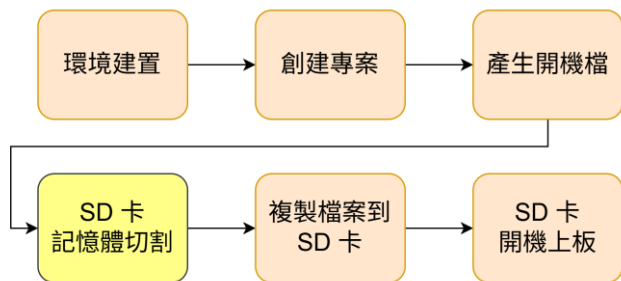
step1



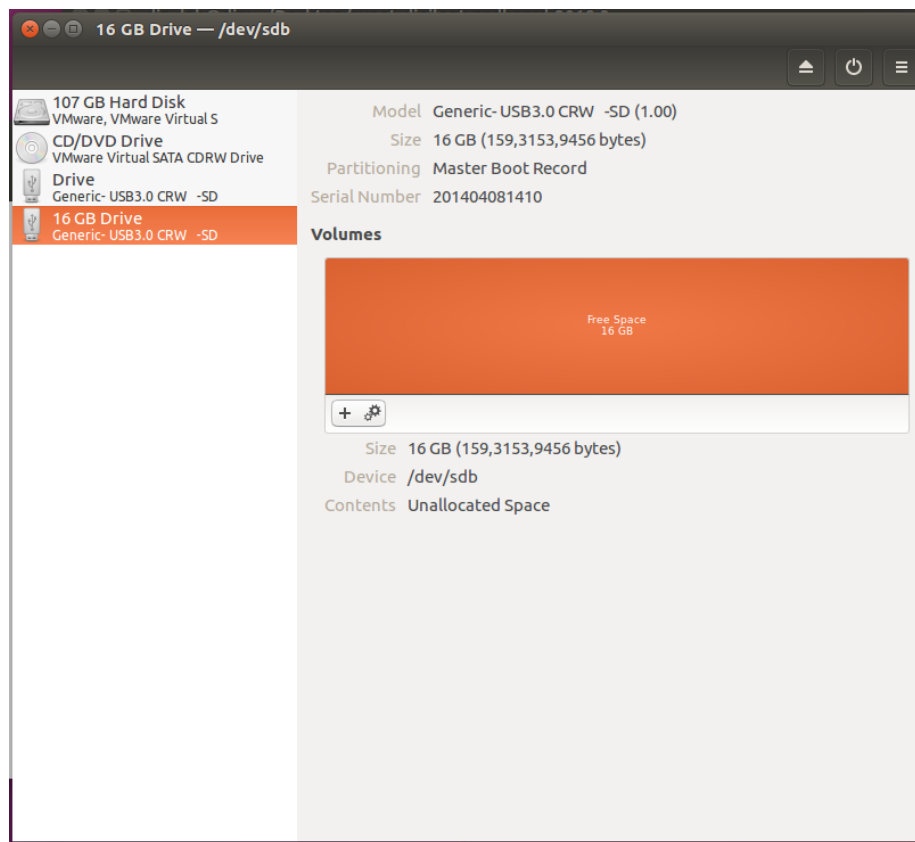
step2



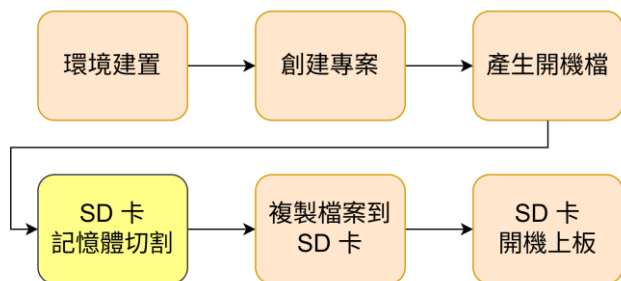
SD card – 記憶體切割



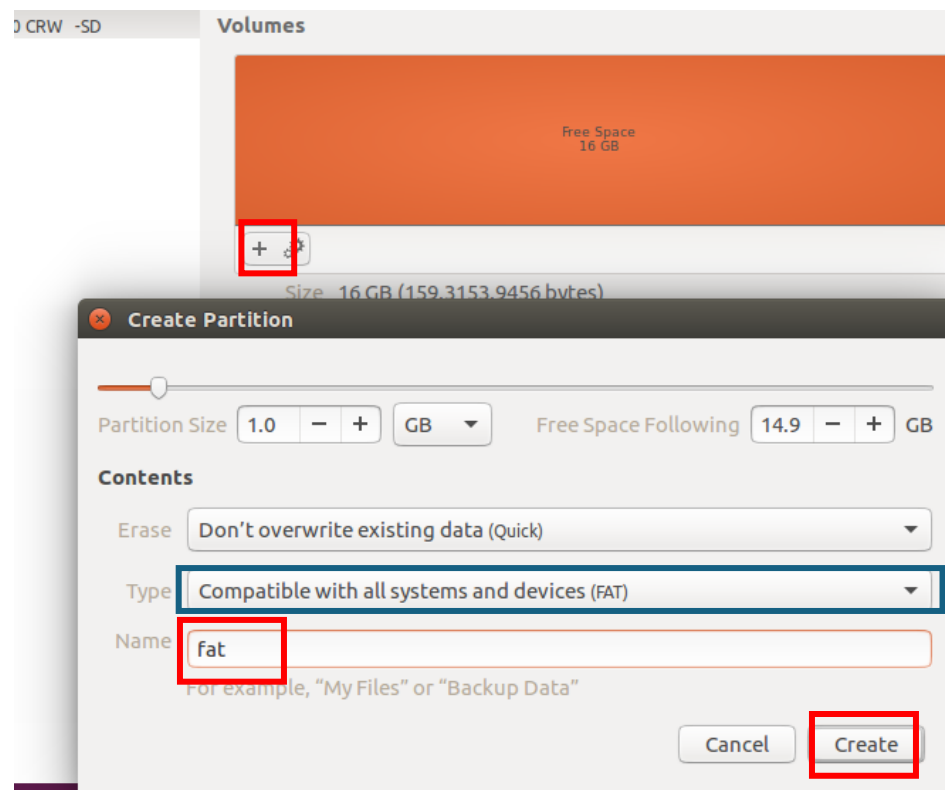
- 畫面就會切換到如右圖



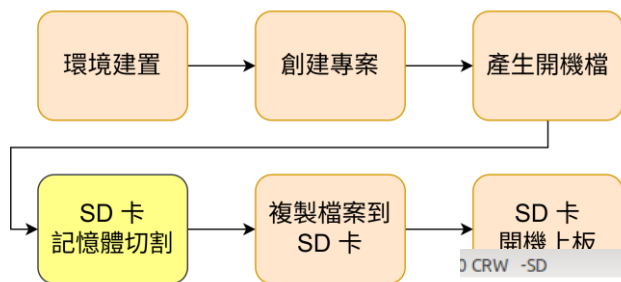
SD card – 記憶體切割



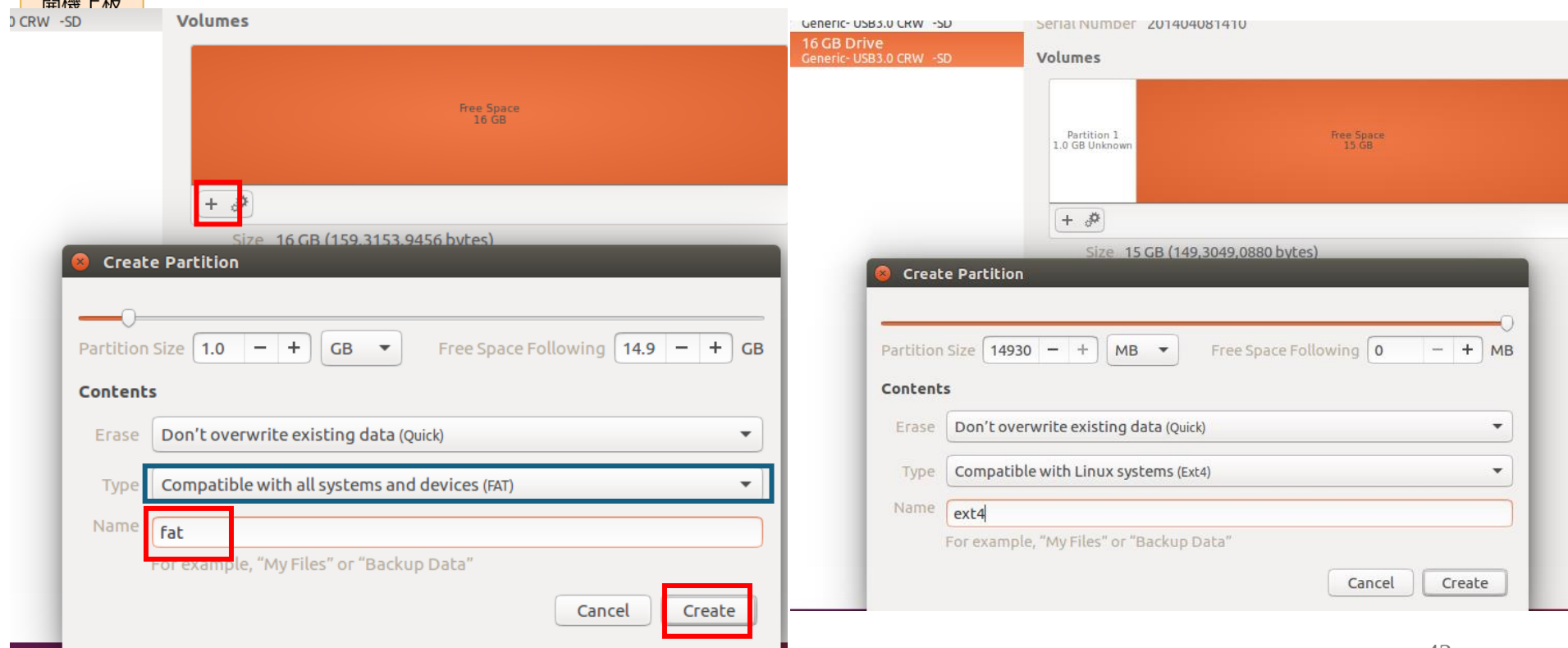
先點選”+”，再依右圖的選項創建
先幫我切 1GB **FAT** : **fat**
剩下的都幫我切成 **EXT4** : **ext4**



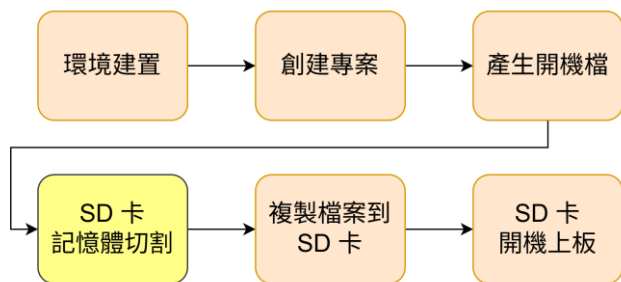
SD card – 記憶體切割



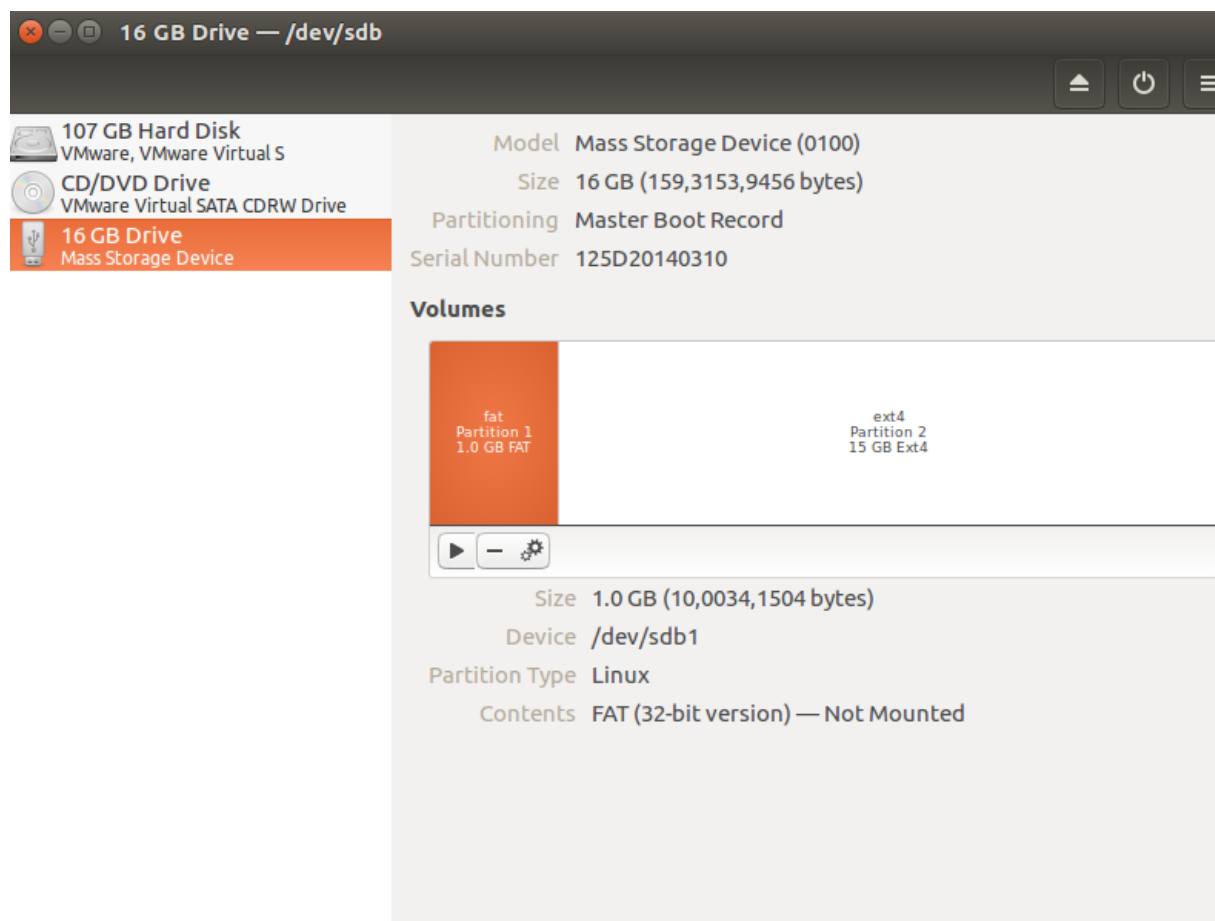
- 先點選“+”，再依右圖的選項創建(如果沒有如預期創建，再把原本的fat,ext4刪除就好，一樣的步驟再做一次)
- 先幫我切 1GB FAT : fat
- 剩下的都幫我切成 EXT4 : ext4



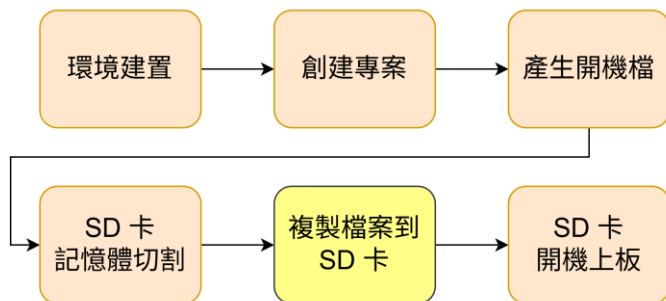
SD card – 記憶體切割



- 切割成功後會如右圖所示
- 注意：fat一定要是partition1，ext4一定要是partition2



複製檔案到SD卡



- 更新變動
- `sudo apt update`
- `sudo apt install bsdtar`

```
ta@ubuntu: ~/Desktop/avnet-digilent-zedboard-2018.3
***** Xilinx Bootgen v2018.3
**** Build date : Nov 15 2018-19:22:29
** Copyright 1986-2018 Xilinx, Inc. All Rights Reserved.

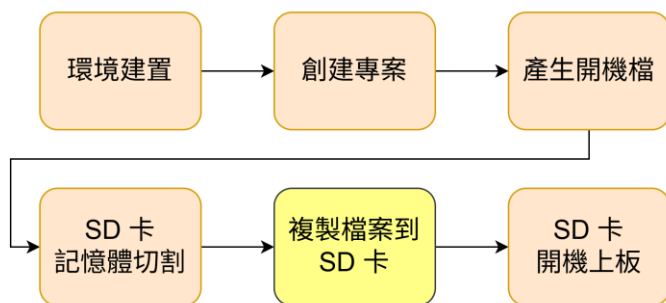
INFO: Binary is ready.
WARNING: Unable to access the TFTPBOOT folder /tftpboot!!!
WARNING: Skip file copy to TFTPBOOT folder!!!
ta@ubuntu:~/Desktop/avnet-digilent-zedboard-2018.3$ sudo apt update
[sudo] password for ta:
Hit:1 http://security.ubuntu.com/ubuntu xenial-security InRelease
Hit:2 http://us.archive.ubuntu.com/ubuntu xenial InRelease
Hit:3 http://us.archive.ubuntu.com/ubuntu xenial-updates InRelease
Hit:4 http://us.archive.ubuntu.com/ubuntu xenial-backports InRelease
Reading package lists... Done
Building dependency tree
Reading state information... Done
47 packages can be upgraded. Run 'apt list --upgradable' to see them.
ta@ubuntu:~/Desktop/avnet-digilent-zedboard-2018.3$ sudo apt install bsdtar
Reading package lists... Done
Building dependency tree
Reading state information... Done
bsdtar is already the newest version (3.1.2-11ubuntu0.16.04.8).
0 upgraded, 0 newly installed, 0 to remove and 47 not upgraded.
ta@ubuntu:~/Desktop/avnet-digilent-zedboard-2018.3$
```

User name

複製檔案到SD卡

- 將下載的作業系統解壓縮到ext4系統碟中，輸入指令：
`sudo bsdtar -xpf ../SOPC_Driver_Lab/ArchLinuxARM-zedboard-latest.tar.gz -C /media/ta/ext4`

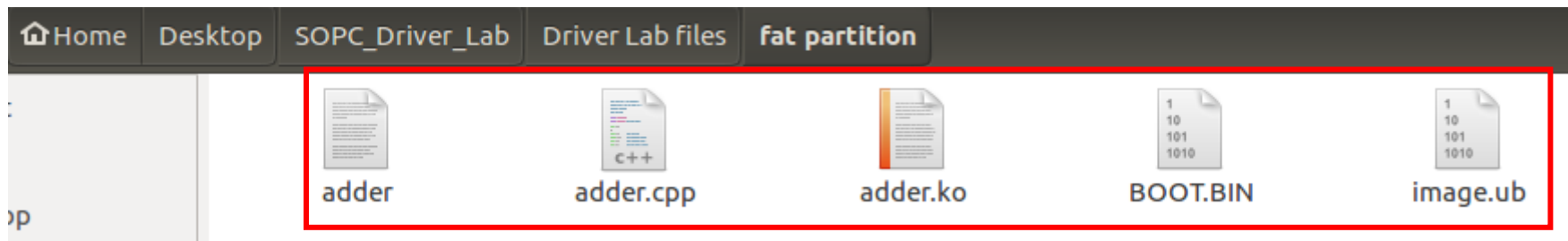
剛切完，它有可能會找不到，只要點開file，點一下ext4就好，再回去terminal打一次上面的指令，這步驟很久正常



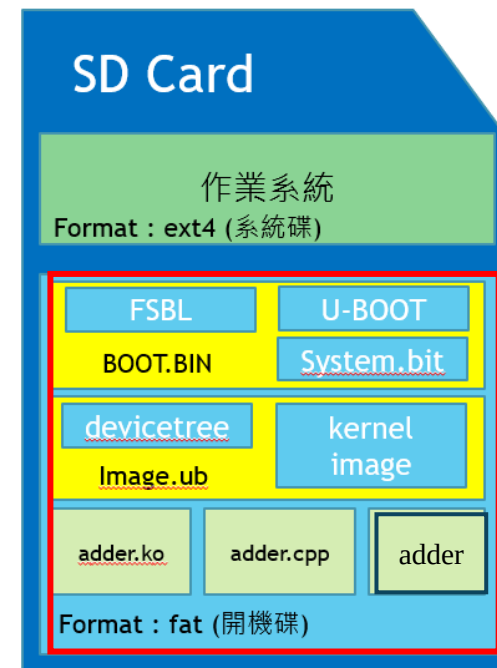
```
ta@ubuntu:~/Desktop/avnet-digilent-zedboard-2018.3$ sudo bsdtar -xpf ../SOPC_Driver_Lab/ArchLinuxARM-zedboard-latest.tar.gz -C /media/ta/ext4
bsdtar: Ignoring malformed pax extended attribute
bsdtar: Ignoring malformed pax extended attribute
bsdtar: Ignoring malformed pax extended attribute
bsdtar: Error exit delayed from previous errors.
```

正常

- 複製這些檔案到SD card fat中(事先都幫大家整理好了)

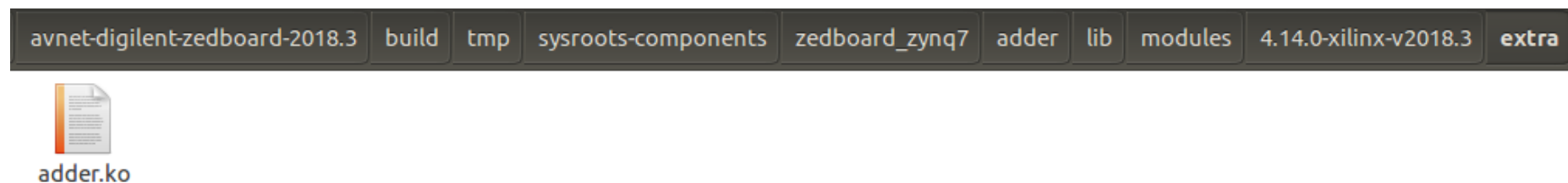


adder, .cpp :本來就有的

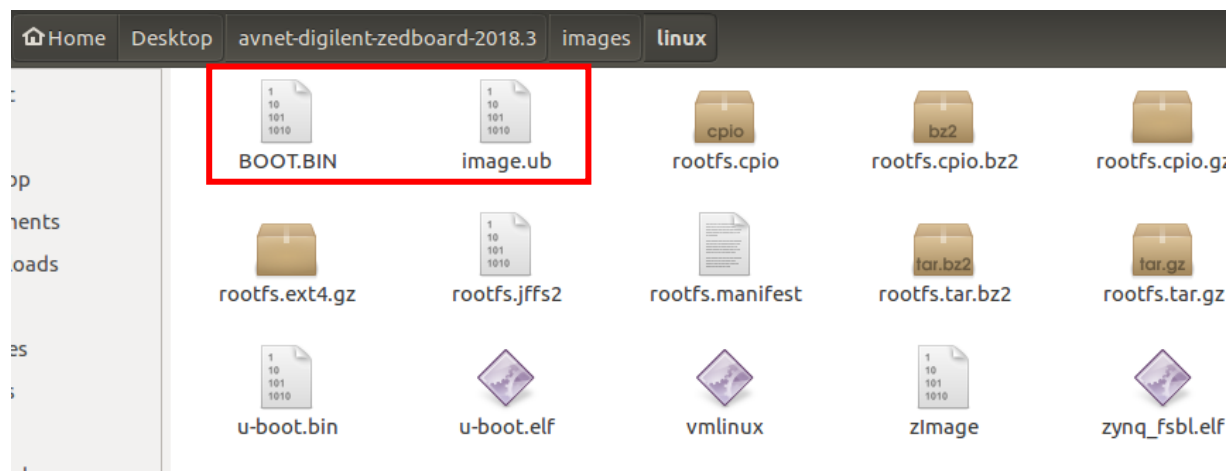


複製檔案到SD卡

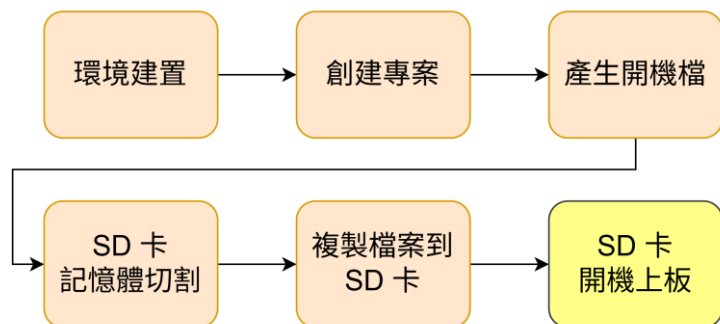
◆ 若想自己抓，提供路徑給各位
.o, .cpp :本來就有的
.ko 在



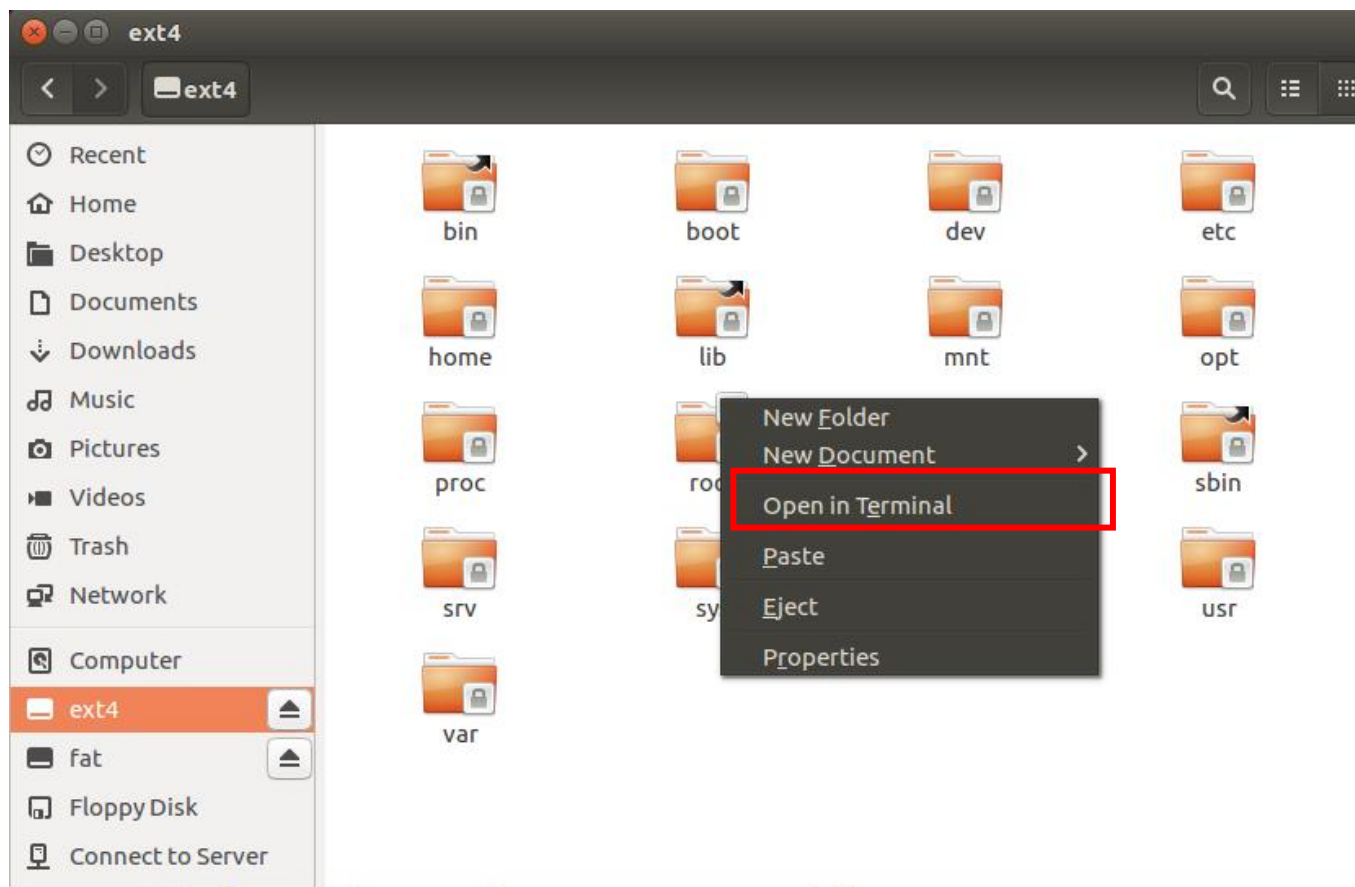
.ub, BOOT 在



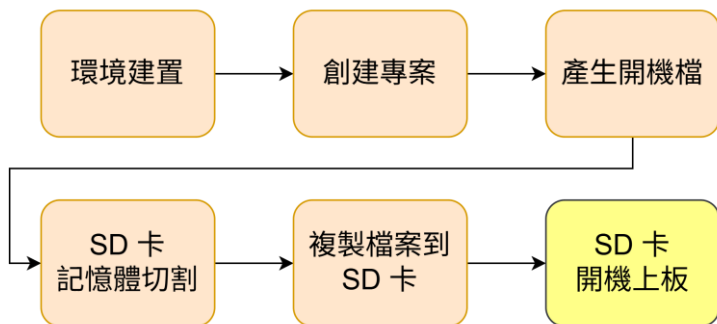
SD卡開機 - 帳號設定



- 點開ext4，並在空白處按右鍵，點選open in terminal



SD卡開機 - 帳號設定



- 注意: 一定要先修改權限才能改檔案內容
- `sudo chown -R ta:ta ~/avnet-digilent-zedboard-2018.3`
- 紅色部分換成自己的 user name

```
ta@ubuntu: /media/ta/ext4
** (gedit:65679): WARNING **: Set document metadata failed: Setting attribute me
tadata::gedit-position not supported
** (gedit:65679): WARNING **: Set document metadata failed: Setting attribute me
tadata::gedit-position not supported
^C
ta@ubuntu:/media/ta/ext4$ sudo chown -R ta:ta ~/avnet-digilent-zedboard-2018.3
ta@ubuntu:/media/ta/ext4$ sudo nautilus

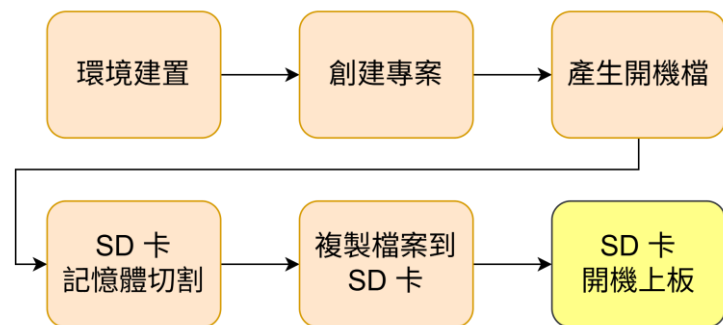
(nautilus:65720): Gtk-WARNING **: Failed to register client: GDBus.Error:org.fre
edesktop.DBus.Error.ServiceUnknown: The name org.gnome.SessionManager was not pr
ovided by any .service files

** (nautilus:65720): CRITICAL **: Another desktop manager in use; desktop window
won't be created

(nautilus:65720): IBUS-WARNING **: The owner of /home/ta/.config/ibus/bus is not
root!

(nautilus:65720): IBUS-WARNING **: Unable to connect to ibus: Unexpected
content trying to read a line
Nautilus-Shares-Message: Called "net usershare info" but it failed: Failed
cute child process "net" (No such file or directory)
```


SD卡開機 - 帳號設定

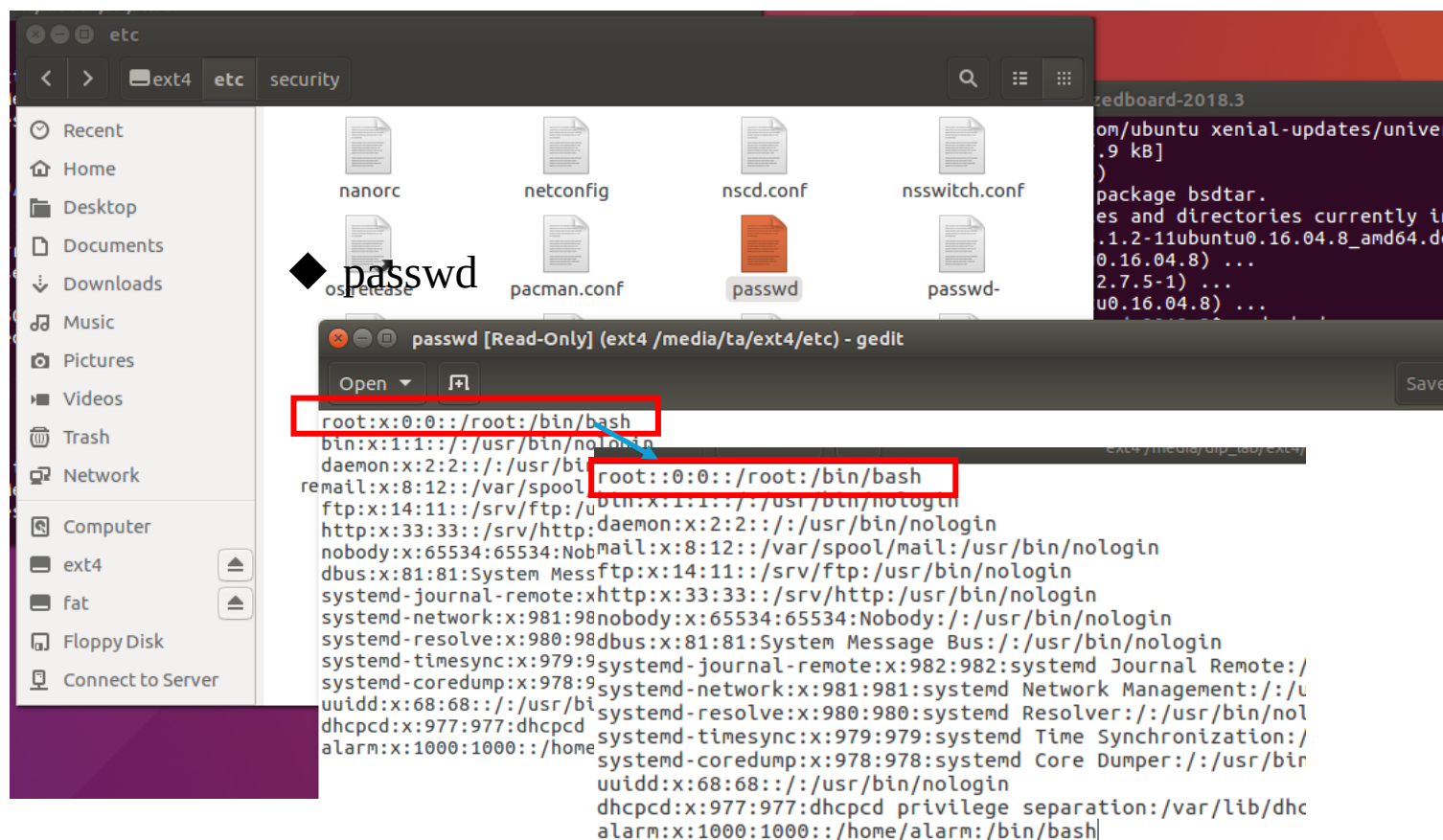


◆ scuretty

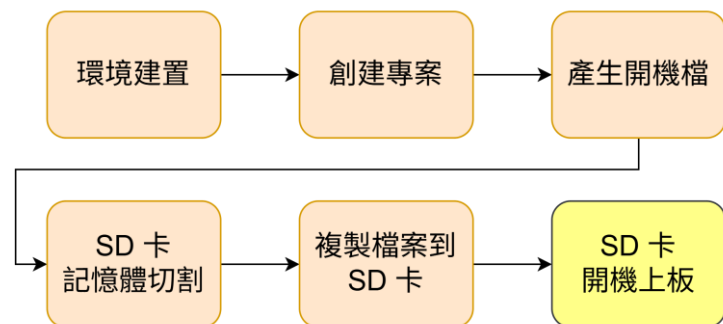
```
# File which lists terminals from which root
# See securetty(5) for details.

console
tty1
tty2
tty3
tty4
tty5
tty6
ttyS0
hvc0
ttyPS0
```

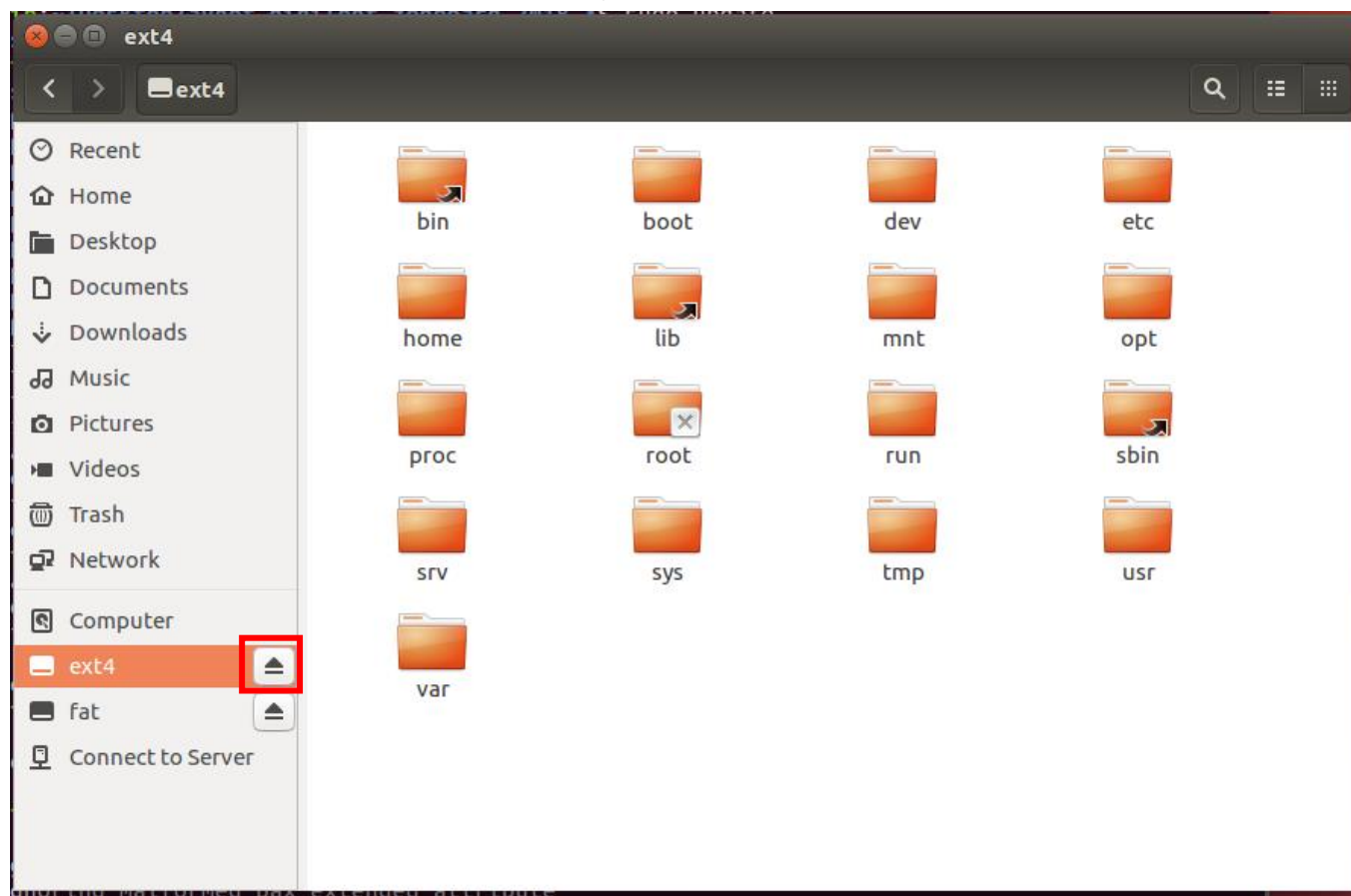
- 輸入指令 : `sudo nautilus`
- 會自動出現一個視窗，在那個視窗中點開`etc/security/`的`scuretty`，`passwd`做修改->`save`



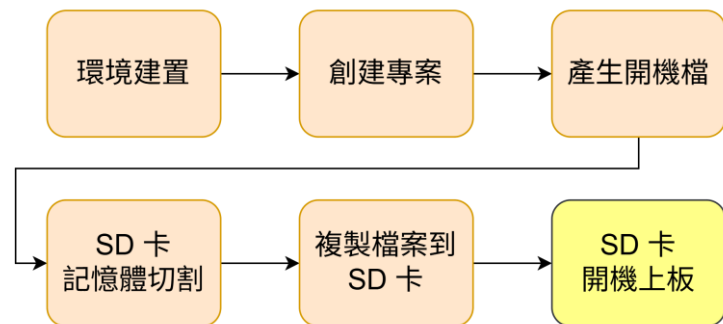
SD卡開機 - 帳號設定



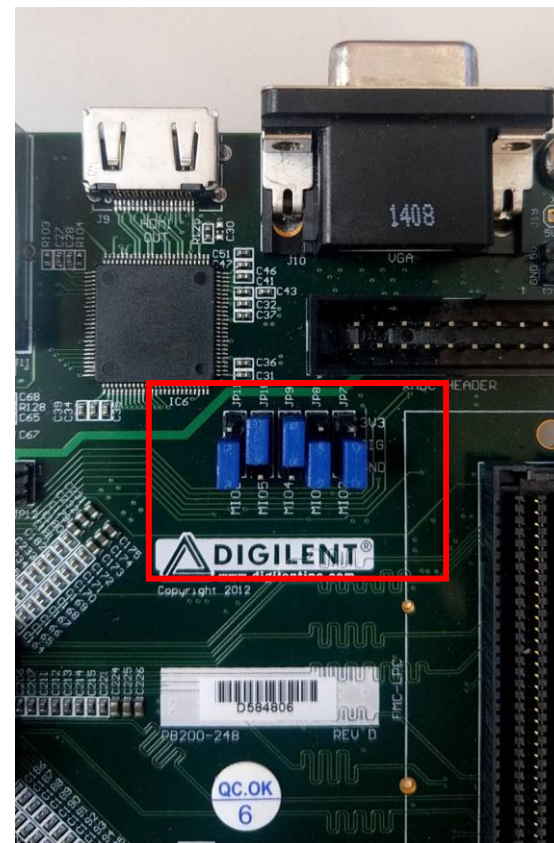
- terminal可以關掉了
- 點選紅框處，此SD卡即燒錄完成，可拔掉了(一定要確認可以拔時才拔，不然寫入SD卡的資料可能會受損)



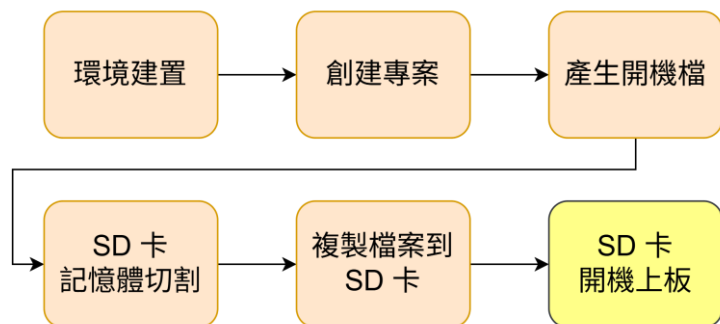
SD卡開機 – 上板



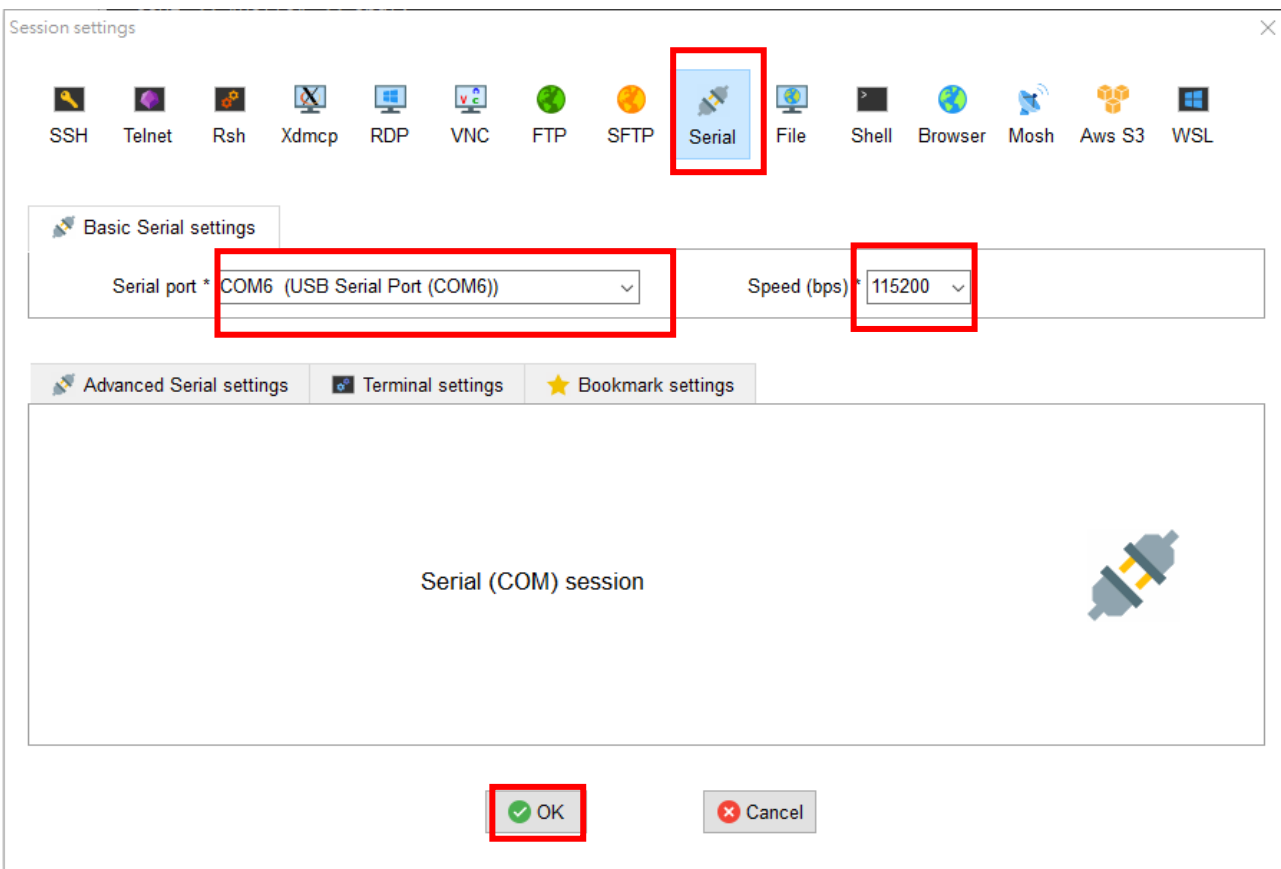
- 將SD卡插到zedboard
- 將zedboard如右圖紅框(SD Card Boot Mode) 所示調整開機順序
- 插上Jtag和Uart的線到電腦



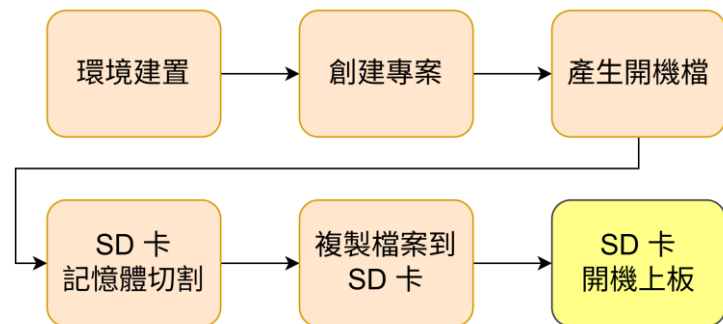
SD卡開機 – 上板



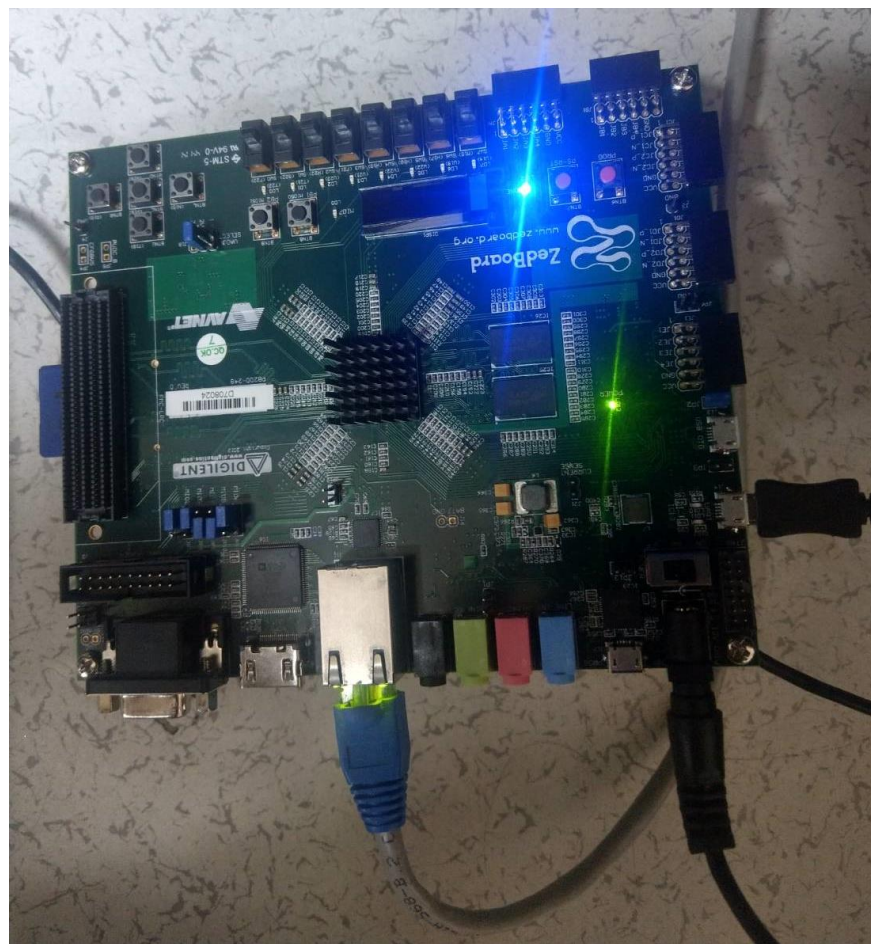
- 插上zedboard電源線，打開電源
- 查裝置管理員port腳
- 開moba，並選對應的port腳，speed 一定要設置!



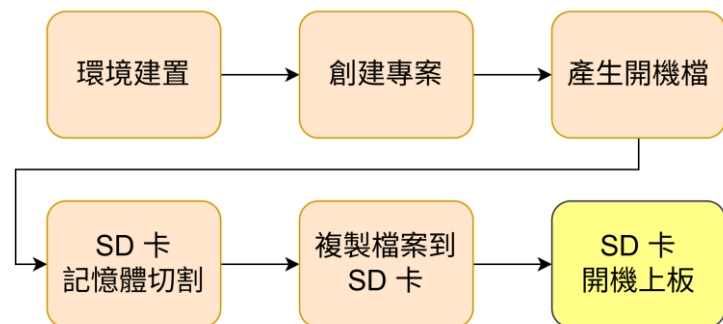
SD卡開機 – 上板



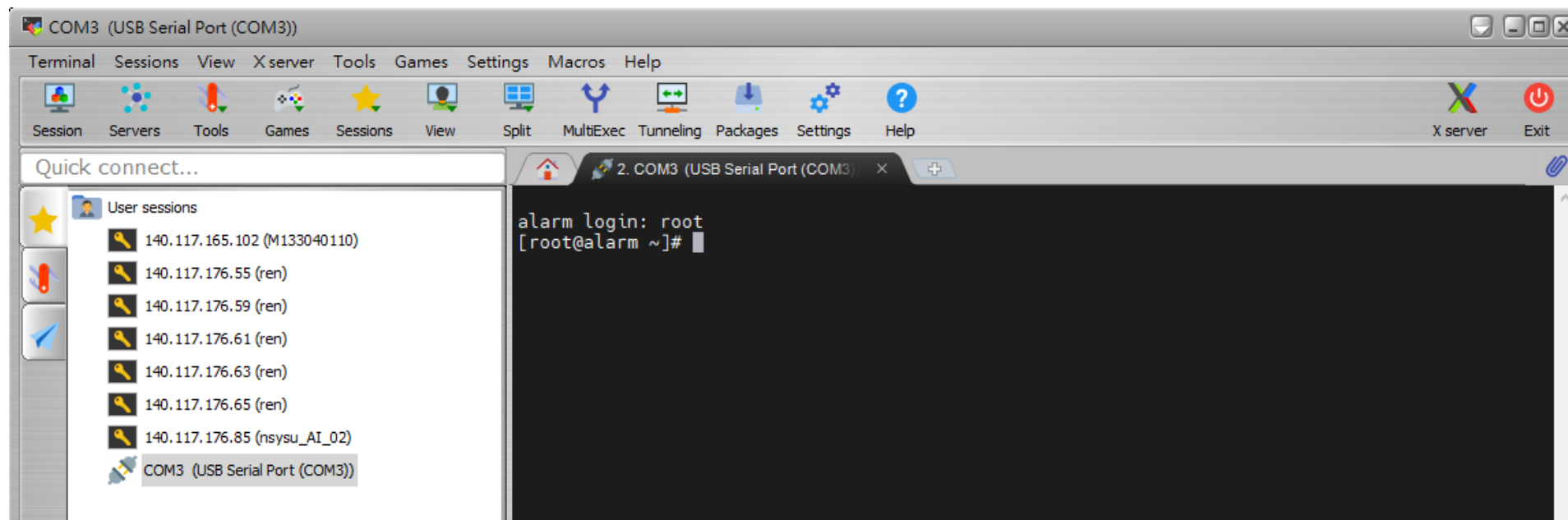
- 有亮藍光，就代表前面petalinux生成的那些檔案是正常的，成功一半了



SD卡開機 – 上板



- **Terminal 直接按Enter**
- 輸入帳號：root（不用打密碼，因為我們剛剛在SD卡帳號設定的地方，已把設密碼的部分刪除）



SD卡開機 – 上板

- 將剛剛切SD卡的第一個partition的資料掛載到fat資料夾中，再將driver掛載到kernel上，依序輸入指令：
- `mkdir fat`
- `mount /dev/mmcblk0p1 fat/`
- `ls -al fat/`
- `ls /dev/`
- `insmod fat/adder.ko`

```
alarm login: root
[root@alarm ~]# ls
[root@alarm ~]# mkdir fat
[root@alarm ~]# mount /dev/mmcblk0p1 fat/
[root@alarm ~]# ls -al fat/
total 8464
drwxr-xr-x 2 root root 4096 Jan 1 1970 .
drwxr-x--- 4 root root 4096 Aug 8 22:31 ..
-rwxr-xr-x 1 root root 4673400 May 19 2022 BOOT.BIN
-rwxr-xr-x 1 root root 9180 Oct 3 2020 adder
-rwxr-xr-x 1 root root 967 Oct 3 2020 adder.cpp
-rwxr-xr-x 1 root root 9532 May 19 2022 adder.ko
-rwxr-xr-x 1 root root 50 May 6 2023 board_setting.sh
-rwxr-xr-x 1 root root 3952428 May 19 2022 image.ub
[root@alarm ~]# ls /dev/
block      memory_bandwidth ram0      tty      tty28    tty48    urandom
bus        mmcblk0         ram1      tty0     tty29    tty49    vcs
char       mmcblk0p1       ram10     tty1     tty3     tty5     vcs1
console    mmcblk0p2       ram11     tty10    tty30    tty50    vcs2
cpu_dma_latency mtd0            ram12     tty11    tty31    tty51    vcs3
disk       mtd0ro          ram13     tty12    tty32    tty52    vcs4
fd         mtd1            ram14     tty13    tty33    tty53    vcs5
full       mtd1ro          ram15     tty14    tty34    tty54    vcs6
gpiochip0  mtd2            ram2      tty15    tty35    tty55    vcsa
io:device0 mtd2ro          ram3      tty16    tty36    tty56    vcsa1
kmsg       mtd3            ram4      tty17    tty37    tty57    vcsa2
log        mtd3ro          ram5      tty18    tty38    tty58    vcsa3
loop-control mtdblock0       ram6      tty19    tty39    tty59    vcsa4
loop0      mtdblock1       ram7      tty2     tty4     tty6     vcsa5
loop1      mtdblock2       ram8      tty20    tty40    tty60    vcsa6
loop2      mtdblock3       ram9      tty21    tty41    tty61    vga_arbiter
loop3      network_latency random          tty22    tty42    tty62    watchdog
loop4      network_throughput shm          tty23    tty43    tty63    watchdog0
loop5      null            snd        tty24    tty44    tty7     zero
loop6      port            stderr     tty25    tty45    tty8
loop7      ptmx            stdin      tty26    tty46    tty9
mem        pts            stdout     tty27    tty47    ttyPS0
[root@alarm ~]# insmod fat/adder.ko
adder: loading out-of-tree module taints kernel.
<1>Hello module world.
adder-master 43c00000.adder: Device Tree Probing
```

SD卡開機 – 上板

- 執行adder application輸入指令：
- `ls /dev/`
- `cd fat/`
- `./adder`

```
adder: loading out-of-tree module taints kernel.
<1>Hello module world.
adder-master 43c00000.adder: Device Tree Probing
adder-master 43c00000.adder: adder at 0x43c00000 mapped to 0xe0b30000
[root@alarm ~]# ls /dev/
adder-master mem pts stdout tty27 tty47 ttyPS0
block memory_bandwidth ram0 tty tty28 tty48 urandom
bus mmcblk0 ram1 tty0 tty29 tty49 vcs
char mmcblk0p1 ram10 tty1 tty3 tty5 vcs1
console mmcblk0p2 ram11 tty10 tty30 tty50 vcs2
cpu_dma_latency mtd0 ram12 tty11 tty31 tty51 vcs3
disk mtd0ro ram13 tty12 tty32 tty52 vcs4
fd mtd1 ram14 tty13 tty33 tty53 vcs5
full mtd1ro ram15 tty14 tty34 tty54 vcs6
gpiochip0 mtd2 ram2 tty15 tty35 tty55 vcsa
iio:device0 mtd2ro ram3 tty16 tty36 tty56 vcsa1
kmsg mtd3 ram4 tty17 tty37 tty57 vcsa2
log mtd3ro ram5 tty18 tty38 tty58 vcsa3
loop-control mtblock0 ram6 tty19 tty39 tty59 vcsa4
loop0 mtblock1 ram7 tty2 tty4 tty6 vcsa5
loop1 mtblock2 ram8 tty20 tty40 tty60 vcsa6
loop2 mtblock3 ram9 tty21 tty41 tty61 vga_arbiter
loop3 network_latency random tty22 tty42 tty62 watchdog
loop4 network_throughput shm tty23 tty43 tty63 watchdog0
loop5 null snd tty24 tty44 tty7 zero
loop6 port stderr tty25 tty45 tty8
loop7 ptmx stdin tty26 tty46 tty9
[root@alarm ~]# ls
fat
[root@alarm ~]# cd fat/
[root@alarm fat]# ls
BOOT.BIN adder adder.cpp adder.ko board_setting.sh image.ub
[root@alarm fat]# ./adder
Put the value to HW register SW→HW
the value is 100

Put the value to HW register SW→HW
the value is 50

Take the value from HW register SW←HW
add : 150
```


SD卡開機 – 上板

- 看到這畫面即是成功

```
add: 150
[root@alarm ~]# cd fat
[root@alarm fat]# ./adder
the value is 100

the value is 50

in output_a
add : 150
in output_b
sub : 50
in output_c
mul : 5000
in output_d
div : 2
```

Thanks for listening
